Facultad de Informática Organizacional

Videojuego de estrategia en tiempo real para contribuir a la promoción de los Objetivos de Desarrollo Sostenible

Trabajo de diploma para optar por el título de
Ingeniero en Ciencias Informáticas

Autor: Oscar de Jesús Pacheco del Pino
Tutores: Dr. C. Juan Antonio Plasencia Soler
Ing. Alain Cruz Jiménez

La Habana, 25 Junio de 2025

Año 67 de la Revolución

DECLARACIÓN DE AUTORÍA

El autor del trabajo de diploma con título "***Videojuego de estrategia en tiempo real para contribuir a la promoción de los Objetivos de Desarrollo Sostenible***" concede a la Universidad de las Ciencias Informáticas los derechos patrimoniales de la investigación, con carácter exclusivo. De forma similar se declara como único autor de su contenido. Para que así conste firma la presente a los 25 días del mes de junio del año 2025.

Oscar de Jesús Pacheco del Pino

_____
Firma del Autor

Juan Antonio Plasencia Soler

_____
Firma del Tutor

Alain Cruz Jiménez

_____
Firma del Tutor

DATOS DE CONTACTO

Juan Antonio Plasencia Soler: Es graduado de Ingeniero Industrial (Universidad Central “Marta Abreu” de las Villas-UCLV, 2003), Máster en Administración de Negocios (Universidad de la Habana, 2008), y Doctor en Ciencias Técnicas (Universidad Tecnológica de la Habana José Antonio Echeverría, 2018). Actualmente es Profesor Titular de la Universidad de las Ciencias Informáticas y miembro de su Consejo Científico, además pertenece al Consejo Técnico Asesor del Ministerio de Educación Superior de Cuba y del Parque Científico Tecnológico de La Habana. Las temáticas de investigación se centran en la sostenibilidad de las organizaciones, la educación para el desarrollo sostenible, y la transformación digital para el cumplimiento de la Agenda 2030. Es coordinador de dos proyectos asociados a programas sectoriales del Ministerio de Ciencia, Tecnología y Medio Ambiente de Cuba.

Alain Cruz Jiménez: Durante su trayectoria en la UCI, ha cumplido diversas tareas en docencia, desarrollo y formación. En la docencia de pregrado en la Facultad 3, impartió Ingeniería de Software 2 y 1, y apoyó a estudiantes con consultas y revisión de trabajos. En CEGEL (ahora CIGE), desarrolló aplicaciones móviles para entidades como SEO WebMAS y MINJUS (iLEX Cuba), y asesoró tesis de pregrado. En la empresa z17 (donde actualmente trabaja), es desarrollador de la aplicación móvil de Apklis. Contribuye al proyecto CEODES, ESODES y BDG, desarrollando soluciones para la educación sostenible y participando en eventos científicos. Y es maestrante en la Maestría en Informática Avanzada.

RESUMEN

El desconocimiento de los Objetivos de Desarrollo sostenible trae consigo múltiples retos en los tiempos actuales, mientras que su conocimiento desvela oportunidades para la mejora desde el nivel personal hasta el global. Buscando una forma de contribuir a la promoción de estos se revela el potencial educativo que poseen los videojuegos, principalmente los del género de estrategia en tiempo real. Se describen las capacidades que posee este tipo de aplicaciones informáticas debido a la posibilidad de crear una narrativa profunda que atraiga a los jugadores y a su vez aplique técnicas para el aprendizaje. Igualmente se documenta el proceso completo de desarrollo de la propuesta de solución desde la selección de las tecnologías a utilizar y la obtención de requisitos funcionales y no funcionales hasta los estándares de codificación, prototipos de interfaces gráficas y tareas de ingeniería. Explicándose en cada momento los elementos que componen al videojuego junto con sus funcionalidades. Por último se demuestra el diseño y la aplicación de las pruebas de software para validar la solución planteada.

PALABRAS CLAVE

Desarrollo, estrategia, objetivos, sostenibilidad, videojuego

ABSTRACT

Not having knowledge about the Sustainable Development Goals brings multiple challenges to current times, while having information about them unveils opportunities for improvement from a personal to a global level. Searching for a way to contribute to the promotion of these, the educational potential for videogames is revealed, mainly of the real time strategy genre. The capabilities that this kind of informatics applications possess are described due to their possibility to create a profound narrative that attracts the players and at the same time applies techniques for learning. Also the full development process of the candidate solution gets documented from the selection of technologies and the functional and non functional requirement elicitation to the coding standards, graphic interface prototypes and engineering tasks. Explaining in every moment the elements that compose the videogame along with their functionalities. Lastly the design and the application of software tests is demonstrated in order to validate the candidate solution.

KEYWORDS

Development, objectives, strategy, sustainability, videogame

TABLA DE CONTENIDOS

INTRODUCCIÓN 11
Marco Teórico 15
CAPÍTULO I: FUNDAMENTOS Y REFERENTES TEÓRICO-METODOLÓGICOS SOBRE VIDEOJUEGOS DE ESTRATEGIA EN TIEMPO REAL 16
I.1 Conceptos Fundamentales 16
I.2 Videojuegos de estrategia en tiempo real 18
I.3 Videojuegos de estrategia en tiempo real para promover los Objetivos de Desarrollo Sostenible 18
I.3.1 Estudio de soluciones similares 19
I.4 Metodologías y herramientas para el desarrollo de Sustainable Conquest 20
I.4.1 Metodología de desarrollo de software 21
I.4.2 Herramienta CASE 22
I.4.3 Lenguaje de modelado 23
I.4.4 Motor de videojuegos 23
I.4.5 Lenguaje de programación 24
I.4.6 Extensión para sistema de diálogos 25
I.4.7 Sistema de control de versiones 26
I.4.8 Software de gestión de proyectos 27
Conclusiones del capítulo 27
CAPÍTULO II: PLANIFICACIÓN, DISEÑO E IMPLEMENTACIÓN DE LA SOLUCIÓN PROPUESTA 28
II.1 Descripción de la propuesta de solución 28
II.2 Planificación 29
II.2.1 Requisitos funcionales 29
II.2.2 Requisitos no funcionales 31
II.2.3 Historias de usuario 31
II.2.4 Estimación de tiempo de desarrollo 40
II.2.5 Plan de iteraciones 41
II.3 Diseño 43
II.3.1 Arquitectura utilizada 43
II.3.3 Prototipos de interfaces gráficas 45
II.3.3 Tarjetas CRC 48
II.4 Implementación 52
II.4.1 Estándares de codificación 52
II.4.2 Patrones utilizados 55
II.4.3 Tareas de Ingeniería 60
Conclusiones del capítulo 68
CAPÍTULO III: VALIDACIÓN DE LA SOLUCIÓN PROPUESTA 69
III.1 Diseño de la validación 69
III.1.1 Estrategia de pruebas 69
III.1.1.1 Pruebas unitarias 69
III.1.1.2 Pruebas de aceptación 70
III.1.1.2 Pruebas de índice de aceptación 70
III.2 Validación 71
III.2.1 Aplicación de pruebas unitarias 71

III.2.3 Aplicación de pruebas de aceptación 72
III.2.3 Aplicación de pruebas de índice de aceptación....73
Conclusiones del capítulo....75
CONCLUSIONES FINALES 76
RECOMENDACIONES....77
REFERENCIAS BIBLIOGRÁFICAS 79

ÍNDICE DE TABLAS

Tabla 1 : Videojuegos RTS relacionados a los ODS [Elaboración propia]. 20
Tabla 2 : Requisitos funcionales [Elaboración propia]. 31
Tabla 3 : Requisitos no funcionales [Elaboración propia]. 31
Tabla 4 : Historia de usuario 1 [Elaboración propia]. 32
Tabla 5 : Historia de usuario 2 [Elaboración propia]. 32
Tabla 6 : Historia de usuario 3 [Elaboración propia]. 33
Tabla 7 : Historia de usuario 4 [Elaboración propia]. 33
Tabla 8 : Historia de usuario 5 [Elaboración propia]. 34
Tabla 9 : Historia de usuario 6 [Elaboración propia]. 34
Tabla 10 : Historia de usuario 7 [Elaboración propia]. 34
Tabla 11 : Historia de usuario 8 [Elaboración propia]. 35
Tabla 12 : Historia de usuario 9 [Elaboración propia]. 35
Tabla 13 : Historia de usuario 10 [Elaboración propia]. 35
Tabla 14 : Historia de usuario 11 [Elaboración propia]. 36
Tabla 15 : Historia de usuario 12 [Elaboración propia]. 36
Tabla 16 : Historia de usuario 13 [Elaboración propia]. 36
Tabla 17 : Historia de usuario 14 [Elaboración propia]. 37
Tabla 18 : Historia de usuario 15 [Elaboración propia]. 37
Tabla 19 : Historia de usuario 16 [Elaboración propia]. 38
Tabla 20 : Historia de usuario 17 [Elaboración propia]. 38
Tabla 21 : Historia de usuario 18 [Elaboración propia]. 39
Tabla 22 : Historia de usuario 19 [Elaboración propia]. 39
Tabla 23 : Historia de usuario 20 [Elaboración propia]. 40
Tabla 24 : Historia de usuario 21 [Elaboración propia]. 40
Tabla 25 : Puntos de estimación [Elaboración propia]. 41
Tabla 26 : Plan de iteraciones [Elaboración propia]. 43
Tabla 27 : Tarjeta CRC Unit [Elaboración propia]. 48
Tabla 28 : Tarjeta CRC Selector [Elaboración propia]. 48
Tabla 29 : Tarjeta CRC Map [Elaboración propia]. 49
Tabla 30 : Tarjeta CRC RTSCamera [Elaboración propia]. 49
Tabla 31 : Tarjeta CRC IDGenerator [Elaboración propia]. 49
Tabla 32 : Tarjeta CRC FormationManager [Elaboración propia]. 49
Tabla 33 : Tarjeta CRC FactionManager [Elaboración propia]. 49
Tabla 34 : Tarjeta CRC TargetManager [Elaboración propia]. 49
Tabla 35 : Tarjeta CRC FogOfWar [Elaboración propia]. 50
Tabla 36 : Tarjeta CRC FogInfo [Elaboración propia]. 50
Tabla 37 : Tarjeta CRC Minimap [Elaboración propia]. 50
Tabla 38 : Tarjeta CRC Stats [Elaboración propia]. 50
Tabla 39 : Tarjeta CRC StatsPerLevel [Elaboración propia]. 50
Tabla 40 : Tarjeta CRC Commander [Elaboración propia]. 50
Tabla 41 : Tarjeta CRC Player [Elaboración propia]. 50
Tabla 42 : Tarjeta CRC VisibilityTileMap [Elaboración propia]. 51
Tabla 43 : Tarjeta CRC AIComponent [Elaboración propia]. 51
Tabla 44 : Tarjeta CRC State [Elaboración propia]. 51
Tabla 45 : Tarjeta CRC Skill [Elaboración propia]. 51
Tabla 46 : Tarjeta CRC SkillData [Elaboración propia]. 51
Tabla 47 : Tarjeta CRC StateManager [Elaboración propia]. 51
Tabla 48 : Tarjeta CRC StateMachine [Elaboración propia]. 52
Tabla 49 : Tarea de Ingeniería 1 [Elaboración propia]. 61
Tabla 50 : Tarea de Ingeniería 2 [Elaboración propia]. 61
Tabla 51 : Tarea de Ingeniería 3 [Elaboración propia]. 61

Tabla 52 : Tarea de Ingeniería 4 [Elaboración propia]. 62
Tabla 53 : Tarea de Ingeniería 5 [Elaboración propia]. 62
Tabla 54 : Tarea de Ingeniería 6 [Elaboración propia]. 62
Tabla 55 : Tarea de Ingeniería 7 [Elaboración propia]. 62
Tabla 56 : Tarea de Ingeniería 8 [Elaboración propia]. 63
Tabla 57 : Tarea de Ingeniería 9 [Elaboración propia]. 63
Tabla 58 : Tarea de Ingeniería 10 [Elaboración propia]. 63
Tabla 59 : Tarea de Ingeniería 11 [Elaboración propia]. 64
Tabla 60 : Tarea de Ingeniería 12 [Elaboración propia]. 64
Tabla 61 : Tarea de Ingeniería 13 [Elaboración propia]. 64
Tabla 62 : Tarea de Ingeniería 14 [Elaboración propia]. 64
Tabla 63 : Tarea de Ingeniería 15 [Elaboración propia]. 65
Tabla 64 : Tarea de Ingeniería 16 [Elaboración propia]. 65
Tabla 65 : Tarea de Ingeniería 17 [Elaboración propia]. 65
Tabla 66 : Tarea de Ingeniería 18 [Elaboración propia]. 66
Tabla 67 : Tarea de Ingeniería 19 [Elaboración propia]. 66
Tabla 68 : Tarea de Ingeniería 20 [Elaboración propia]. 66
Tabla 69 : Tarea de Ingeniería 21 [Elaboración propia]. 67
Tabla 70 : Listado de estudiantes probadores beta [Elaboración propia]. 73
Tabla 71 : Cuestionario de Satisfacción del Usuario - Sustainable Conquest [Elaboración Propia]. 74

ÍNDICE DE FIGURAS

Figura 1 : SceneTree de Godot [Elaboración propia]....44
Figura 2 : Barra de estado [Elaboración propia]. 45
Figura 3 : Barra de estado para Comandantes [Elaboración propia]....45
Figura 4 : Contenedor de unidades [Elaboración propia]. 46
Figura 5 : Barra de habilidades [Elaboración propia]. 46
Figura 6 : Descripción de habilidad [Elaboración propia]....46
Figura 7 : Contenedor de recursos [Elaboración propia]. 46
Figura 8 : Minimapa [Elaboración propia]. 47
Figura 9 : Cola de entrenamiento [Elaboración propia]....47
Figura 10 : Menú de pausa [Elaboración propia]. 47
Figura 11 : Estructura de archivos jerárquica [Elaboración propia]. 53
Figura 12 : Estructura de archivos relacional [Elaboración propia]....54
Figura 13 : Uso de identaciones en GDScript [Elaboración propia]....54
Figura 14 : Clase Player con comentarios de documentación [Elaboración propia]. 54
Figura 15 : Documentación autogenerada de clase Player [Elaboración propia]....55
Figura 16 : Ejemplo de patrón experto en la clase NavigationTileMap [Elaboración propia]. 57
Figura 17 : Ejemplo de patrón Bajo acoplamiento en clase FogInfo [Elaboración propia]. 57
Figura 18 : Ejemplo del patrón bajo acoplamiento en la clase VisibilityTileMap [Elaboración propia]. 58
Figura 19 : Uso del patrón Singleton mediante autoloads en Godot [Elaboración propia]. 59
Figura 20 : Panel de señales en Godot [Elaboración propia]. 59
Figura 21 : Panel de conexión de señales en Godot [Elaboración propia]. 59
Figura 22 : Conexión de señales a través de código [Elaboración propia]....60
Figura 23 : Uso del patrón Strategy en la máquina de estados de la clase Unit [Elaboración propia]. 60
Figura 24 : Casos de prueba para funcionalidad hasEnoughResources() de la clase Player [Elaboración propia]. 72
Figura 25 : Resultado de pruebas para la clase Player [Elaboración propia]. 72

AVAL DEL CLIENTE

Universidad de las Ciencias Informáticas

La Habana, 15 de junio del 2025

ACTA DE ACEPTACIÓN

Asunto: Carta de aceptación de la solución informática "Videojuego de estrategia en tiempo real para contribuir a la promoción de los Objetivos de Desarrollo Sostenible"

Por medio de la presente, me permito informar que, tras la revisión y evaluación correspondientes, se acepta formalmente la solución informática desarrollada bajo el nombre "Videojuego de estrategia en tiempo real para contribuir a la promoción de los Objetivos de Desarrollo Sostenible", desarrollada por el estudiante Oscar de Jesús Pacheco del Pino como parte de su Trabajo de Diploma.

Esta aplicación forma parte del proyecto "Ecosistema de Ciencia, Tecnología e Innovación para contribuir al cumplimiento de los Objetivos de Desarrollo Sostenible desde las Instituciones de Educación Superior", adscrito al Programa Sectorial de Ciencia, Tecnología e Innovación: "Gestión de Ciencia, Tecnología e Innovación para el Desarrollo Sostenible" del Ministerio de Ciencia Tecnología y Medioambiente (CITMA). Reconocemos que la solución presentada cumple con los requisitos funcionales y técnicos establecidos, y consideramos que su implementación aportará significativamente al fortalecimiento de la gestión educativa y al avance institucional en materia de sostenibilidad.

Los miembros del proyecto agradecemos la dedicación y profesionalismo en el desarrollo de esta herramienta, y reiteramos nuestro compromiso de apoyar su integración y uso en beneficio de la comunidad universitaria y del logro de los Objetivos de Desarrollo Sostenible.

Atentamente,

Dr. C. Juan Antonio Plasencia Soler
Jefe de Proyecto
Universidad de las Ciencias Informáticas

Juan Antonio Plasencia Soler

Firmado digitalmente por Juan Antonio Plasencia Soler
DN: OID.0.9.2342.19200300.100.1.1=juanps, CN=Juan Antonio Plasencia Soler, SERIALNUMBER=T142289, G=Juan Antonio, SN=Plasencia Soler, C=CU
Razón: Soy el autor de este documento
Ubicación:
Fecha: 2025.06.11 03:56:55+02'00'
Foxit PDF Reader Versión: 2025.1.0

INTRODUCCIÓN

La sostenibilidad se define como la capacidad de satisfacer las necesidades del presente sin comprometer la capacidad de las futuras generaciones para satisfacer sus propias necesidades, abarcando la interrelación entre el desarrollo económico, la inclusión social y la protección del medio ambiente (WCED, 1987). Este concepto implica un enfoque holístico que busca un equilibrio entre estos tres pilares para garantizar un futuro viable para todos. Las dimensiones de la sostenibilidad se dividen comúnmente en tres categorías principales: la dimensión económica, que se centra en el crecimiento económico y la creación de riqueza, asegurando que las actividades económicas sean sostenibles a largo plazo mediante un uso eficiente de los recursos y prácticas comerciales responsables (Stiglitz et al., 2018); la dimensión social, que se refiere a la equidad social, la inclusión y el bienestar humano, enfatizando la importancia de garantizar que todas las personas tengan acceso a recursos básicos, educación y salud, promoviendo así la justicia social (Naciones Unidas, 2015, p14); y la dimensión ambiental, enfocada en la protección y conservación del medio ambiente, que incluye la gestión sostenible de los recursos naturales, la reducción de la contaminación y la conservación de la biodiversidad (European Union, 2024), aunque en algunas literaturas se incluyen la paz y alianzas como dimensiones.

La educación para el desarrollo sostenible es fundamental en la promoción de un futuro equitativo y sostenible. Este tipo de educación no solo busca transmitir conocimientos, sino que también fomenta habilidades y actitudes que permiten a las personas tomar decisiones informadas y responsables en relación con su entorno y la sociedad. Según la UNESCO (2017; 2020), la Educación para el Desarrollo Sostenible (EDS) es un enfoque integral que fomenta la adquisición de conocimientos, competencias, valores y actitudes necesarios para construir sociedades justas y sostenibles. Esto incluye la integración transversal de temas como el cambio climático y la equidad social en los sistemas educativos, así como el empoderamiento de los ciudadanos para participar activamente en la toma de decisiones responsables hacia un futuro sostenible. Además, la EDS contribuye directamente a la consecución de los ODS, especialmente a la meta 4.7 del ODS 4, que busca garantizar que todas las personas adquieran conocimientos teóricos y prácticos necesarios para promover el desarrollo

sostenible, incluyendo la educación sobre derechos humanos, igualdad de género y estilos de vida sostenibles (Naciones Unidas, 2015; UNESCO, 2020). Al empoderar a las personas con conocimientos sobre sostenibilidad, se les proporciona la capacidad de abordar desafíos globales como el cambio climático, la pobreza y la desigualdad, lo que subraya la importancia de integrar la sostenibilidad en todos los niveles educativos.

La educación es un derecho humano fundamental y un motor clave para el desarrollo sostenible. Sin embargo, persisten brechas críticas: en 2021, 244 millones de niños y jóvenes estaban sin escolarizar, una cifra que refleja desigualdades arraigadas en geografía, género y nivel socioeconómico (Global Education Monitoring Report Team, 2022, p. 12). Estas brechas se intensifican en crisis humanitarias: 222 millones de niños afectados por conflictos o desastres naturales requieren apoyo educativo urgente, dejando a uno de cada cinco adolescentes sin ejercer su derecho a la educación (UNESCO, 2023). A esto se suma la crisis de aprendizaje: el 70% de los niños en países de ingresos bajos y medios no adquieren competencias básicas en lectura, y el 55% fracasa en matemáticas, perpetuando ciclos de pobreza (UNESCO, 2023; Banco Mundial, 2025)". La pandemia de COVID-19 exacerbó las desigualdades educativas preexistentes: 463 millones de niños en edad escolar (31% del total global) no tuvieron acceso a educación a distancia durante los cierres escolares, según UNICEF (2020). Esta cifra incluye 13 millones de niños en América Latina, donde países como México reportaron que el 78.6% de las familias enfrentaron dificultades para continuar la educación en casa, principalmente por falta de dispositivos y conectividad. Por lo tanto, abordar estas brechas es esencial para lograr el Objetivo de Desarrollo Sostenible 4 (ODS 4), que busca garantizar una educación inclusiva, equitativa y de calidad para todos.

El ODS 4 busca garantizar una educación inclusiva, equitativa y de calidad, promoviendo oportunidades de aprendizaje durante toda la vida para todos. En Cuba, el sistema educativo ha sido históricamente un pilar fundamental del desarrollo social y económico, logrando altas tasas de alfabetización y acceso a la educación. Cuba mantiene uno de los sistemas educativos más inclusivos de América Latina: según la UNESCO (2023), el 99.7% de su población adulta está alfabetizada, y la educación es gratuita en todos los niveles, desde preescolar hasta superior, tal como lo establece la Constitución (2019, Artículo 73). Este

modelo se sustenta en una inversión del 12.9% del PIB en educación, la más alta de la región. Sin embargo, persisten desafíos como la escasez de materiales didácticos, la obsolescencia tecnológica y las limitaciones de conectividad, que afectan la calidad educativa en contextos de crisis económica (Banco Mundial, 2023; Mesa-Lago, 2022). Además, el Primer Informe Nacional Voluntario de Cuba (2021) destaca que, aunque se han realizado esfuerzos para mejorar la formación docente y los recursos educativos, persisten desafíos en infraestructura educativa y acceso a tecnologías actualizadas, lo que impacta en la calidad del sistema. Por lo tanto, aunque Cuba ha logrado avances significativos hacia el ODS 4, aún queda un camino por recorrer para asegurar una educación de calidad y equitativa para todos.

Los videojuegos se han posicionado como herramientas pedagógicas innovadoras para abordar desafíos de sostenibilidad. Un ejemplo destacado es 'Eco', un juego multijugador que simula la construcción de una civilización sostenible en un ecosistema virtual. Los jugadores deben gestionar recursos naturales, negociar políticas colectivas y enfrentar consecuencias ambientales (como la extinción de especies por sobreexplotación), lo que fomenta una comprensión práctica de la interdependencia entre desarrollo humano y salud planetaria (Wu & Lee, 2020). Otro referente es '*Fate of the World*', que desafía a los jugadores a implementar políticas climáticas globales basadas en datos científicos. Este juego ha demostrado mejorar la conciencia sobre la complejidad de la gobernanza ambiental, especialmente en contextos de incertidumbre y cooperación internacional (Flood et al., 2024). Estos videojuegos no solo entretienen, sino que desarrollan habilidades clave para los ODS: pensamiento crítico, colaboración y toma de decisiones basada en evidencia (Plass et al., 2020). Además, títulos como '*Cities: Skylines*' (usado en aulas para enseñar planificación urbana sostenible) y '*Frostpunk*' (que explora dilemas éticos en crisis climáticas) amplían el aprendizaje experiencial mediante mecánicas que vinculan acciones individuales con impactos sistémicos (Hawlitschek et al., 2022; 11 Bit Studios, 2024).

Los juegos de estrategia en tiempo real (RTS) han evolucionado para integrar temáticas vinculadas a los Objetivos de Desarrollo Sostenible (ODS), ofreciendo marcos interactivos que promueven la reflexión sobre gestión responsable de recursos y equilibrio socioambiental. *Age of Empires IV* (Relic Entertainment, 2021) permite a los jugadores explorar el desarrollo

histórico de civilizaciones, donde decisiones como la adopción de tecnologías renovables en expansiones recientes reflejan el ODS 7 (Energía Asequible) y el ODS 11 (Ciudades Sostenibles). Por su parte, *Stellaris* (Paradox Interactive, 2016), a través de actualizaciones como Nemesis (2021), incorpora dilemas éticos sobre explotación planetaria y conservación de ecosistemas alienígenas, alineándose con el ODS 12 (Producción Responsable) y el ODS 15 (Vida de Ecosistemas Terrestres). Ambos títulos demuestran cómo los RTS pueden fomentar conciencia crítica sobre desafíos globales mediante mecánicas de planificación estratégica y gestión de recursos, aunque sin un enfoque explícito en la Agenda 2030.

El conocimiento de los ODS trae consigo varios factores positivos como son la alineación con la Agenda Global, mejora en la toma de decisiones, ventajas competitivas, acceso a recursos, fomento de la responsabilidad social y ambiental, oportunidades de innovación y emprendimiento sostenible e impulso de políticas verdes de los gobiernos (R3crea, 2022). Por otra parte su desconocimiento trae consigo una desalineación con tendencias internacionales, oportunidades perdidas, riesgo en la toma de decisiones, uso inadecuado de recursos, falta de preparación para el futuro, desconexión con la naturaleza, degradación ambiental, empeoramiento del cambio climático, problemas de salud pública y conflictos sociales (EcologíaVerde, 2024). En nuestro país de forma general y en la misma Universidad de las Ciencias Informáticas se observa un desconocimiento de los ODS por lo que se desea encontrar una solución que contribuya a la promoción de los mismos

No se ha encontrado ningún videojuego específico desarrollado en Cuba que se centre exclusivamente en la enseñanza de la sostenibilidad. Sin embargo, hay iniciativas educativas que utilizan tecnologías digitales y videojuegos como herramientas para aumentar la conciencia sobre la sostenibilidad y el medio ambiente, añadiéndole a esto la pasión que sienten muchas personas por los videojuegos y la gran capacidad que poseen los videojuegos de estrategia en tiempo real para el Aprendizaje Basado en Juegos (ABJ), se presentó la posibilidad de desarrollar un videojuego que contribuya a la promoción de los objetivos de desarrollo sostenible.

Marco Teórico

El presente marco teórico se orienta a fundamentar y contextualizar la investigación en torno al **problema científico**: ¿Cómo contribuir a la promoción de los Objetivos de Desarrollo Sostenible? En este sentido, se plantea como **objeto de estudio** los videojuegos de estrategia, focalizando **el campo de acción** de videojuegos de estrategia en tiempo real para contribuir a la promoción de los Objetivos de Desarrollo Sostenible, por considerarlos instrumentos idóneos para fomentar el conocimiento y la sensibilización sobre estos objetivos globales. **El objetivo general del estudio** es desarrollar un videojuego de estrategia en tiempo real que contribuya a la promoción de los Objetivos de Desarrollo Sostenible. Para alcanzar este propósito, se han definido tres **objetivos específicos** que comprenden: establecer los referentes teórico-metodológicos relacionados con la promoción de los Objetivos de Desarrollo Sostenible, realizar el análisis, diseño e implementación de la solución propuesta, y validar el funcionamiento del videojuego a través de la aplicación de métricas de calidad y pruebas de software. Finalmente, la **metodología de investigación** se fundamenta en un enfoque integral que combina métodos empíricos, como la observación apoyada en el análisis comparativo de otros videojuegos, con métodos teóricos, como la modelación, garantizando así una comprensión y análisis del fenómeno desde múltiples perspectivas.

CAPÍTULO I: FUNDAMENTOS Y REFERENTES TEÓRICO-METODOLÓGICOS SOBRE VIDEOJUEGOS DE ESTRATEGIA EN TIEMPO REAL

Este capítulo tiene como objetivo establecer los fundamentos teóricos y metodológicos que sustentan el desarrollo de videojuegos de estrategia en tiempo real (RTS) como herramientas para promover los Objetivos de Desarrollo Sostenible (ODS). Se abordarán los conceptos básicos de los RTS, su evolución histórica y su potencial como medios educativos y de concienciación. Además, se analizará cómo estos videojuegos pueden integrarse en contextos académicos, específicamente en la Universidad de las Ciencias Informáticas (UCI), para promover los objetivos de desarrollo sostenible. El capítulo se estructura en cuatro epígrafes principales: (1) una revisión teórica de los RTS, (2) su aplicación en la promoción de los ODS, (3) su implementación en la UCI y (4) las metodologías y herramientas para su desarrollo. A través de este análisis, se busca proporcionar un marco conceptual para justificar la pertinencia de la investigación y su contribución al campo de la ingeniería y la sostenibilidad.

I.1 Conceptos Fundamentales

Para la comprensión del artículo se definen a continuación los conceptos fundamentales de la investigación:

Videojuego: Un videojuego es un sistema digital interactivo que integra mecánicas de juego (reglas y objetivos), narrativa (historia o contexto) y tecnología (gráficos, sonido, inteligencia artificial) para generar experiencias inmersivas (Rogers, 2020). Su diseño se fundamenta en principios como la interactividad (respuestas en tiempo real a las acciones del jugador), objetivos estructurados (ej.: resolver puzzles o combatir enemigos) y retroalimentación (recompensas que guían el comportamiento del usuario) (Johnson & Lee, 2021). Además, la adaptabilidad, mediante dificultad ajustable o algoritmos de aprendizaje automático, permite personalizar la experiencia según el estilo de juego individual (Chen et al., 2022).

Sostenibilidad: principio rector que busca armonizar el progreso humano con la preservación del medio ambiente y la equidad social, asegurando que las acciones actuales no comprometan el bienestar de las generaciones futuras. Según el Programa de las Naciones Unidas para el Desarrollo (PNUD), la sostenibilidad implica "un desarrollo que equilibra las dimensiones económica, social y ambiental, garantizando que los recursos naturales se utilicen

de manera responsable y que las comunidades prosperen de manera inclusiva" (PNUD, 2021). Reconoce que los sistemas naturales, sociales y económicos están interconectados y que su equilibrio es esencial para la supervivencia y prosperidad a largo plazo. La sostenibilidad también requiere un cambio de paradigma hacia modelos de producción y consumo más responsables, así como la adopción de políticas y prácticas que fomenten la resiliencia frente a desafíos globales como el cambio climático, la pérdida de biodiversidad y la desigualdad social (Elkington, 2021).

Objetivos de Desarrollo Sostenible: conjunto de 17 metas globales adoptadas por las Naciones Unidas en 2015 como parte de la Agenda 2030 para el Desarrollo Sostenible. Estos objetivos buscan abordar los desafíos más urgentes que enfrenta la humanidad, como la pobreza, la desigualdad, el cambio climático, la degradación ambiental, la paz y la justicia. Según las Naciones Unidas, los ODS son "un plan maestro para lograr un futuro sostenible para todos, equilibrando las dimensiones económica, social y ambiental del desarrollo" (Naciones Unidas, 2020). Cada objetivo está interconectado y se enfoca en áreas específicas, como la erradicación del hambre (ODS 2), la educación de calidad (ODS 4), la igualdad de género (ODS 5), el trabajo decente (ODS 8) y la acción por el clima (ODS 13), entre otros. Los ODS no solo son un llamado a la acción para los gobiernos, sino también para las empresas, las organizaciones de la sociedad civil y los ciudadanos, promoviendo la colaboración global para construir un mundo más justo, equitativo y sostenible (Sachs et al., 2021).

Educación para el desarrollo sostenible (EDS): enfoque educativo integral que busca empoderar a las personas, comunidades y sociedades para tomar decisiones informadas y responsables que contribuyan a un futuro sostenible. Según la UNESCO, la EDS es "un proceso de aprendizaje que permite a las personas adquirir los conocimientos, las competencias, las actitudes y los valores necesarios para construir un futuro sostenible" (UNESCO, 2020). Este enfoque no se limita a la transmisión de conocimientos, sino que también busca desarrollar habilidades prácticas, como el pensamiento crítico, la resolución de problemas y la colaboración, que son esenciales para enfrentar los desafíos globales interconectados. La EDS abarca las tres dimensiones clave de la sostenibilidad. La EDS puede abordar temas como la conservación de los recursos naturales, la equidad social, la justicia económica y la participación ciudadana, fomentando una comprensión profunda de cómo estas áreas están relacionadas (Sterling, 2016). Además, promueve valores como la

responsabilidad, la solidaridad y el respeto por la diversidad, que son fundamentales para construir sociedades más equitativas y sostenibles (UNESCO, 2020).

Aprendizaje basado en juegos (ABJ): metodología educativa que utiliza juegos, ya sean digitales o analógicos, como herramientas pedagógicas para facilitar la adquisición de conocimientos, habilidades y actitudes. Según Plass et al. (2020), el ABJ "se fundamenta en el diseño de entornos lúdicos que estimulan la motivación intrínseca, la exploración autónoma y la resolución de problemas en contextos complejos, replicando dinámicas del mundo real" (p. 15). Esta metodología integra elementos como la narrativa inmersiva, la retroalimentación adaptativa y la colaboración en tiempo real, mecanismos que incrementan la participación activa y la retención de conceptos (Deterding et al., 2020). Además, el ABJ promueve habilidades del siglo XXI, como el pensamiento crítico, la creatividad y la gestión de recursos, adaptándose a las necesidades individuales mediante sistemas de inteligencia artificial (UNESCO, 2021). Estudios recientes demuestran que su aplicación en entornos educativos formales mejora significativamente el rendimiento académico y la capacidad de transferir conocimientos a situaciones prácticas (European Council, 2022).

I.2 Videojuegos de estrategia en tiempo real

Los videojuegos de estrategia en tiempo real (RTS) son un género que combina elementos de planificación, gestión de recursos y toma de decisiones en un entorno dinámico y competitivo. Desde su aparición en la década de 1990, los RTS han evolucionado significativamente, incorporando mecánicas complejas y narrativas profundas que los convierten en herramientas versátiles para el entretenimiento y la educación (WIRED, 2021). A nivel internacional, títulos como StarCraft y Age of Empires han sentado las bases del género, demostrando su capacidad para fomentar habilidades cognitivas como el pensamiento crítico, la resolución de problemas y la gestión del tiempo (Martínez, 2023). En nuestro país, aunque el desarrollo de RTS ha sido más limitado, existen iniciativas que buscan aprovechar este género para fines educativos y sociales.

I.3 Videojuegos de estrategia en tiempo real para promover los Objetivos de Desarrollo Sostenible

Los videojuegos de estrategia en tiempo real (RTS) han demostrado ser herramientas efectivas para abordar temas complejos como la sostenibilidad, gracias a su capacidad para simular

escenarios reales y fomentar la toma de decisiones informadas (Richard et al., 2020). Estos juegos permiten a los jugadores experimentar con sistemas complejos, como economías, ecosistemas y sociedades, lo que los convierte en plataformas ideales para explorar estrategias de desarrollo sostenible. Por ejemplo, en Civilization VI, los jugadores deben equilibrar el desarrollo económico con la protección del medio ambiente, tomando decisiones que afectan la calidad del aire, la disponibilidad de recursos y la estabilidad política (UNESCO, 2020). De igual forma, en Frostpunk, los jugadores gestionan una ciudad en un mundo postapocalíptico, tomando decisiones difíciles sobre la asignación de recursos, la justicia social y la supervivencia a largo plazo (Zagal et al., 2022).

En el ámbito internacional, varios juegos han demostrado el potencial de los RTS para promover los Objetivos de Desarrollo Sostenible (ODS). Este tipo de juegos educa a los jugadores sobre los desafíos de la sostenibilidad y a su vez fomenta la colaboración y la toma de decisiones colectivas.

A nivel nacional, aunque el uso de RTS para promover los ODS es incipiente, existen proyectos que buscan integrar estos principios en el diseño de videojuegos educativos. El potencial educativo y de concienciación de los RTS es amplio. Su capacidad para simular sistemas complejos y fomentar la toma de decisiones estratégicas los convierte en herramientas ideales para abordar temas relacionados con la sostenibilidad, la economía, la política y la historia. Pueden ser utilizados para enseñar conceptos de economía circular, energías renovables y justicia social, proporcionando a los jugadores una comprensión profunda de los desafíos y oportunidades relacionados con los ODS. Además fomentan habilidades cognitivas como el pensamiento crítico, la resolución de problemas y la gestión del tiempo. Estas habilidades son transferibles a contextos reales, lo que convierte a los RTS en herramientas valiosas para la educación formal e informal (WIRED, 2021).

I.3.1 Estudio de soluciones similares

Para profundizar en posibles soluciones existentes y probar la factibilidad de la solución propuesta se listaron cinco videojuegos de estrategia en tiempo real con la mayor relación a los ODS posible. Los cuales se encuentran a continuación.

Videojuego	Temática	Conexión a los ODS

Civilization VI	Gestión de civilizaciones históricas y evolución cultural.	Gestión de recursos, el desarrollo tecnológico y las decisiones ambientales.
They Are Billions	Supervivencia en un mundo postapocalíptico con gestión de recursos.	Industria, innovación e infraestructura.
Cataclismo	Construcción de fortalezas y defensa contra hordas de criaturas.	Ciudades sostenibles.
Frostpunk	Supervivencia en una sociedad postapocalíptica con gestión de recursos limitados.	Optimización de recursos y diseño estratégico de infraestructuras. Energía asequible y no contaminante.
Age of Empires II: Definitive Edition	Desarrollo histórico de civilizaciones.	Gestión de recursos naturales y avances tecnológicos.

Tabla 1: Videojuegos RTS relacionados a los ODS [Elaboración propia].

Mientras que los videojuegos listados tienen relaciones con los ODS, estas son de forma indirecta. Hasta la fecha actual de la investigación no se han encontrado antecedentes que intenten promover los ODS, lo que resalta la necesidad de desarrollar proyectos como Sustainable Conquest, que entretengan, eduquen y sensibilicen sobre los ODS. Además, la implementación de este RTS en fortalecerá las competencias técnicas de los estudiantes y contribuirá a la formación de profesionales comprometidos con el desarrollo sostenible.

I.4 Metodologías y herramientas para el desarrollo de Sustainable Conquest

Este epígrafe se dedica a la sistematización de los fundamentos teórico-metodológicos asociados a las tecnologías y herramientas utilizadas en el desarrollo de Sustainable Conquest, con el fin de lograr los resultados planteados en el objetivo general de la investigación. Se analizan las metodologías, motores de videojuegos, lenguajes de programación y herramientas CASE seleccionadas, explicando su relevancia y justificación en el contexto del proyecto. La elección de estas tecnologías se basó en su capacidad para

facilitar un desarrollo ágil, eficiente y alineado con los Objetivos de Desarrollo Sostenible (ODS), así como en su adaptabilidad a un entorno académico con recursos y tiempo limitados. A continuación, se describen los elementos clave que sustentaron el proceso de desarrollo, destacando su contribución al proyecto.

I.4.1 Metodología de desarrollo de software

Una metodología de desarrollo de software es un conjunto de principios, prácticas y procesos sistemáticos que guían la planificación, diseño, implementación, prueba y mantenimiento de sistemas de software. Según Sommerville (2021), las metodologías de desarrollo de software "proporcionan un marco estructurado para gestionar la complejidad de los proyectos de software, asegurando que se cumplan los requisitos del cliente y se entregue un producto de alta calidad dentro del tiempo y el presupuesto establecidos" (p. 32). Estas metodologías pueden variar desde enfoques tradicionales, como el modelo en cascada, que sigue una secuencia lineal de fases, hasta enfoques ágiles, como Scrum o Extreme Programming (XP), que promueven la flexibilidad, la colaboración y la entrega iterativa de funcionalidades (Schwaber & Sutherland, 2020). La elección de una metodología depende de factores como el tamaño del proyecto, la claridad de los requisitos, la experiencia del equipo y las necesidades del cliente. Estas buscan mejorar la eficiencia, reducir riesgos y garantizar que el software final sea funcional, escalable y alineado con las expectativas del usuario (Pressman & Maxim, 2021).

Para el desarrollo del videojuego, se utilizó la metodología de desarrollo de software *Extreme Programming* (XP), es una metodología ágil de desarrollo de software que enfatiza la flexibilidad, la colaboración y la entrega continua de valor al cliente. Formulada originalmente por Kent Beck en 1999 (Beck, 1999), sigue siendo relevante en la actualidad por su enfoque en prácticas técnicas rigurosas y adaptabilidad a entornos dinámicos (Cockburn, 2020). Esta metodología se centra en prácticas como la programación en parejas, donde dos desarrolladores trabajan en conjunto para mejorar la calidad del código y reducir errores; el desarrollo guiado por pruebas (TDD), que prioriza la escritura de pruebas automatizadas antes del código funcional para garantizar coherencia; y la integración continua, que asegura actualizaciones frecuentes y detección temprana de fallos. Además, XP fomenta la retroalimentación constante con los usuarios, integrando al cliente en el proceso de desarrollo mediante iteraciones cortas (generalmente 1-2 semanas), lo que permite ajustar requisitos en

tiempo real y alinear el producto con las necesidades del. XP promueve un entorno de trabajo colaborativo donde los desarrolladores y los clientes trabajan juntos para definir las características del software, facilitando una mejor alineación entre las necesidades del usuario y el producto final.

A pesar de ser una metodología de desarrollo de software más tradicional y no específicamente de videojuegos su elección se justifica por varias razones. En primer lugar, la naturaleza dinámica del desarrollo de videojuegos requiere una metodología que permita adaptarse rápidamente a los cambios en el diseño y las mecánicas del juego, algo que XP facilita a través de su enfoque iterativo e incremental (Schwaber & Sutherland, 2020). Además, la colaboración estrecha entre desarrollador y cliente es fundamental para garantizar que el contenido del juego esté alineado con los ODS; XP fomenta esta colaboración mediante las reuniones diarias. Por último, el énfasis de XP en la calidad del software, a través de pruebas continuas y refactorización, asegura que Sustainable Conquest sea un producto funcional y a su vez un excelente recurso educativo.

I.4.2 Herramienta CASE

Las herramientas CASE (*Computer-Aided Software Engineering*) son software diseñado para apoyar el desarrollo, mantenimiento y gestión de sistemas de software, automatizando y optimizando diversas etapas del ciclo de vida del mismo. Según Pressman y Maxim (2021), las herramientas CASE son "aplicaciones que facilitan el modelado, diseño, implementación y documentación de sistemas de software, mejorando la productividad y la calidad del producto final" (p. 356). Estas herramientas abarcan una amplia gama de funcionalidades, como la creación de diagramas con UML (*Unified Modeling Language*), la gestión de requisitos, la generación de código, la prueba de software y la documentación automatizada (Sommerville, 2021). Su uso es fundamental en entornos de desarrollo complejos, ya que permiten a los equipos trabajar de manera más eficiente, reducir errores y garantizar la coherencia en todo el proceso de desarrollo.

Se utilizó Visual Paradigm en su versión 8.0 como herramienta CASE para apoyar el diseño, modelado y documentación del sistema. Visual Paradigm es una plataforma integral que ofrece un conjunto de herramientas para el análisis y diseño de software, incluyendo diagramas UML, gestión de requisitos, modelado de bases de datos y prototipado de interfaces de usuario. Su versatilidad y facilidad de uso lo convierten en una opción ideal para proyectos de ingeniería

de software, especialmente en entornos académicos y de investigación (Visual Paradigm, 2023).

Visual Paradigm permite a los desarrolladores crear y gestionar modelos de software de manera eficiente. Algunas de sus características principales incluyen su soporte de diagramas UML como diagramas de clases, secuencias, casos de uso y actividades. Permite definir, organizar y rastrear los requisitos del sistema, asegurando que todas las funcionalidades estén alineadas con los objetivos del proyecto. Su capacidad para realizar modelado de bases de datos, incluyendo herramientas para diseñar esquemas de bases de datos y generar *scripts* SQL automáticamente. Su prototipado de interfaces, ofreciendo funcionalidades para crear wireframes y prototipos de interfaces de usuario, lo que es especialmente útil en el diseño de videojuegos (Visual Paradigm, 2023). Además de su colaboración en equipo, incluyendo funciones de colaboración en línea, lo que permite a los miembros del equipo trabajar de manera simultánea en el diseño y documentación del sistema.

I.4.3 Lenguaje de modelado

El lenguaje de modelado que se utilizó es UML (*Unified Modeling Language*) en su versión 2.5.1, este es un lenguaje de modelado visual estandarizado que se utiliza para representar, diseñar y documentar sistemas de software. Según Fowler (2018), UML es "una herramienta esencial para visualizar la estructura, el comportamiento y las interacciones de un sistema, permitiendo a los equipos de desarrollo comunicar ideas complejas de manera clara y precisa" (p. 45). UML se utiliza como herramienta fundamental para modelar los diferentes aspectos del sistema, desde la arquitectura general hasta los detalles de implementación, asegurando que se tenga una comprensión común del diseño y funcionalidad del software.

UML trae consigo varias ventajas como una comunicación efectiva, proporcionada por ser un lenguaje común para que desarrolladores y *stakeholders* para la discusión y comprensión del diseño del sistema. Documentación clara al servir como documentación visual, facilitando el mantenimiento y la evolución del software. Detección temprana de errores al modelar el sistema antes de la implementación, posibillitando la identificación y corrección de problemas de diseño en etapas tempranas.

I.4.4 Motor de videojuegos

Un motor de videojuegos es un conjunto de herramientas y tecnologías que permiten el desarrollo, la creación y la ejecución de videojuegos. Estos motores proporcionan una

infraestructura que incluye gráficos, física, sonido, inteligencia artificial y otras funcionalidades necesarias para crear un entorno interactivo y dinámico (Gregory, 2020). Los motores de videojuegos facilitan el trabajo de los desarrolladores al ofrecer bibliotecas y APIs que simplifican la implementación de características complejas, permitiendo a los creadores centrarse en el diseño y la jugabilidad del producto final (Hutchins, 2021).

Godot Engine es un motor de código abierto que ha ganado popularidad en la comunidad de desarrollo de videojuegos por su flexibilidad y facilidad de uso. En su versión 4.4.1, Godot presenta mejoras significativas en el rendimiento gráfico, incluyendo soporte para gráficos 3D avanzados y un sistema de iluminación global que mejora la calidad visual de los juegos (Godot Engine, 2023). Además, Godot 4.4.1 incluye un sistema de *scripting* mejorado basado en GDScript, así como soporte para otros lenguajes como C# y VisualScript, lo que permite a los desarrolladores elegir el lenguaje que mejor se adapte a sus necesidades (Godot Engine, 2023). La interfaz de usuario ha sido optimizada para facilitar la creación y edición de escenas, lo que resulta en un flujo de trabajo más eficiente (Godot Engine, 2023).

La elección de Godot Engine para desarrollar Sustainable Conquest se justifica por su naturaleza de código abierto permite a los desarrolladores personalizar y adaptar el motor según las necesidades específicas del proyecto sin incurrir en costos adicionales. Además, las capacidades gráficas avanzadas de Godot 4.4.1 son ideales para crear un videojuego atractivo que capte la atención de los estudiantes y promueva los ODS de manera efectiva. Por último, la comunidad activa y el amplio soporte documental de Godot facilitan el aprendizaje y la resolución de problemas durante el proceso de desarrollo, lo cual es crucial en un entorno académico donde los recursos pueden ser limitados (Godot Engine, 2023).

I.4.5 Lenguaje de programación

Un lenguaje de programación es un sistema formal de comunicación que permite a los desarrolladores escribir instrucciones precisas para que una computadora ejecute tareas específicas. Según Sebesta (2020), un lenguaje de programación es "un conjunto de reglas sintácticas y semánticas que definen cómo se escriben y estructuran los programas, permitiendo a los humanos expresar algoritmos y lógica de manera comprensible para las máquinas". Estos lenguajes pueden ser de alto nivel, como Python o Java, que se acercan más al lenguaje humano y son más fáciles de leer y escribir, o de bajo nivel, como el lenguaje ensamblador, que está más cerca del código máquina y ofrece un control más directo sobre el

hardware (Aho et al., 2021). Los lenguajes de programación son herramientas esenciales en el desarrollo de software, ya que permiten crear aplicaciones, sistemas operativos, videojuegos y otras soluciones tecnológicas. Además, evolucionan constantemente para adaptarse a las necesidades emergentes, como la programación orientada a objetos, la programación funcional y el desarrollo de inteligencia artificial (Morris, 2021).

Se utilizó GDScript como lenguaje de programación principal. Este es un lenguaje de *scripting* dinámico y de alto nivel diseñado específicamente para el motor de videojuegos Godot Engine. Su sintaxis es similar a Python, lo que lo hace accesible para programadores de todos los niveles, desde principiantes hasta expertos. GDScript está optimizado para la creación de videojuegos, ofreciendo una integración nativa con las funcionalidades de Godot, como la gestión de escenas, nodos y recursos, lo que permite un desarrollo ágil y eficiente (Godot Engine, 2023).

GDScript posee una sintaxis sencilla y legible. Su estructura es fácil de aprender, lo que reduce la curva de aprendizaje y acelera el proceso de desarrollo. Está diseñado para trabajar de manera nativa con Godot, facilitando la manipulación de objetos del motor, como nodos, escenas y recursos. Aunque es un lenguaje interpretado, está optimizado para ejecutarse eficientemente dentro de Godot y trae funcionalidades básicas comunes para el desarrollo de juegos, brindando una superioridad ante otros lenguajes para la misma tarea, haciéndolo adecuado para juegos 2D y 3D de mediana complejidad. Cuenta con una comunidad activa y documentación extensa que proporcionan recursos, tutoriales y soporte, lo que facilita la resolución de problemas y la implementación de nuevas funcionalidades (Linietsky & Manzur, 2020).

La elección de GDScript para Sustainable Conquest se justifica por varias razones. Su compatibilidad con Godot Engine, la rapidez en el desarrollo, la simplicidad de GDScript que permite implementar mecánicas complejas como la gestión de recursos y la inteligencia artificial en menos tiempo comparado con otros lenguajes como C# o C++. Su facilidad para la creación de prototipos rápidos y la iteración constante, factor esencial para probar y ajustar las mecánicas del juego. Por último la comunidad y recursos disponibles permiten un desarrollo más eficiente.

I.4.6 Extensión para sistema de diálogos

Para proveer el sistema de diálogos se utilizó Dialogic en su versión 2.0, esta es una potente extensión (*plugin*) gratuita y de código abierto para el motor Godot Engine, diseñada específicamente para simplificar la creación y gestión de diálogos en videojuegos. Desarrollada por Emilio Coppola, esta herramienta se ha convertido en una de las soluciones más populares para implementar sistemas de conversaciones en juegos 2D y 3D, gracias a su interfaz intuitiva y su integración nativa con Godot (Coppola, 2023). Dialogic proporciona un editor gráfico dentro del propio Godot, permitiendo crear diálogos mediante nodos y conexiones visuales, sin necesidad de escribir código manualmente. Esto facilita la creación de conversaciones complejas con múltiples ramificaciones, respuestas condicionales y eventos personalizados (Godot Engine Documentation, 2023). Permite importar/exportar diálogos en formatos como JSON o CSV, lo que facilita la colaboración con escritores o la integración con herramientas externas de narrativa (Coppola, 2023).

I.4.7 Sistema de control de versiones

El sistema de control de versiones que se utiliza en el desarrollo de la propuesta de solución es GIT en su versión 2.43.0. Git es un sistema de control de versiones distribuido ampliamente utilizado para gestionar cambios en el código fuente durante el desarrollo de software. Según Chacon y Straub (2014), este permite a los desarrolladores "rastrear modificaciones en archivos, coordinar el trabajo en equipo y mantener un historial completo de cambios, facilitando la colaboración y la reversión a versiones anteriores en caso de errores". A diferencia de los sistemas de control de versiones centralizados, Git opera de manera distribuida, lo que significa que cada desarrollador tiene una copia completa del repositorio, incluyendo su historial de cambios, lo que permite trabajar sin conexión y sincronizar cambios posteriormente (Loeliger & Ponuthorai, 2022).

Git utiliza un modelo basado en *snapshots* (instantáneas) en lugar de diferencias entre archivos, lo que lo hace eficiente en términos de almacenamiento y rendimiento. Además, ofrece funcionalidades avanzadas como ramificaciones (*branching*) y fusiones (*merging*), que permiten a los equipos trabajar en múltiples características o correcciones de manera simultánea sin interferir entre sí (Loeliger & Ponuthorai, 2022). Estas ramas pueden luego fusionarse en una rama principal, como *main* o *master*, una vez que se completan y prueban las funcionalidades.

Otra característica clave de Git es su gestión de problemas (*issues*) y la implementación continua (CI/CD) (Chacon & Straub, 2014). Convirtiéndolo en una herramienta esencial para el desarrollo moderno de software, tanto en proyectos pequeños como en grandes equipos distribuidos.

I.4.8 Software de gestión de proyectos

Se establece Github Projects como software de gestión de proyectos, esta es una herramienta integrada en GitHub que permite a los equipos de desarrollo gestionar y organizar tareas, *issues* y *pull requests* de manera eficiente. GitHub Projects es una solución flexible para la planificación y seguimiento del progreso de proyectos de software, ofreciendo una vista visual del flujo de trabajo y las prioridades del equipo (GitHub, 2022). Esta herramienta es especialmente útil para equipos que siguen metodologías ágiles, como XP, ya que permite crear tableros personalizados con columnas que representan diferentes etapas del desarrollo, como "Por hacer", "En progreso" y "Terminado". GitHub Projects incluye características como la automatización de tareas, que mueve automáticamente las tareas entre columnas según ciertos criterios, y la integración con *issues* y *pull requests*, lo que facilita el seguimiento del progreso y la trazabilidad de las tareas (GitHub, 2022). Además, ofrece vistas personalizables, como listas de tareas y gráficos de progreso, que permiten adaptar la herramienta a las necesidades específicas del equipo. GitHub Projects es una herramienta clave para gestionar el flujo de trabajo, planificar tareas, seguir *bugs* y automatizar procesos, mejorando la colaboración y la eficiencia del equipo.

Conclusiones del capítulo

1. Los videojuegos de estrategia en tiempo real (RTS) son herramientas versátiles que combinan entretenimiento y educación, demostrando su potencial para fomentar habilidades cognitivas como el pensamiento crítico, la resolución de problemas y la gestión del tiempo, teniendo un papel relevante en la promoción de los ODS al permitr que los jugadores experimenten con estrategias que equilibran el desarrollo económico, social y ambiental.
2. Se realizó y justificó la selección de las herramientas a utilizar para el desarrollo del videojuego Sustainable Conquest

CAPÍTULO II: PLANIFICACIÓN, DISEÑO E IMPLEMENTACIÓN DE LA SOLUCIÓN PROPUESTA

Este capítulo tiene como objetivo presentar el proceso de análisis, diseño e implementación del videojuego Sustainable Conquest, detallando las decisiones técnicas y metodológicas que sustentan su desarrollo. Este se estructura en la descripción de la propuesta de solución y tres ejes principales guíados por la metodología XP: (1) planificación, donde se definen los requisitos y la organización del proyecto; (2) diseño, que abarca la arquitectura del sistema y las mecánicas de juego; y (3) implementación, que describe la codificación y las herramientas utilizadas. A través de este análisis, se busca proporcionar una visión clara y sistemática de cómo se materializó el proyecto. Se evidencia desde la obtención de requisitos hasta la implementación final, respaldado por tablas, modelos, y diagramas que garantizan la calidad del producto.

II.1 Descripción de la propuesta de solución

Sustainable Conquest es un videojuego de estrategia en tiempo real en un mundo fantástico y futurista en el cual se narra la historia entre dos paises. Aeloria, la cima tecnológica y de la innovación, un pueblo pacifista y fiel al uso de energías limpias y desarrollo sostenible. Por otra parte, Grimhold, tierra oscura altamente militarista, única en descubrir el poder que trae consigo la radiactividad, gobernada por la corrupción y la tiranía.

Existen fricciones constantes entre ambos paises debido a los recursos naturales existentes en Aeloria y en medio de una feria de sostenibilidad Grimhold bombardea la capital, dejando pocos sobrevivientes, que al sufrir este acontecimiento deciden rebelarse, pero no de forma vengativa contra quien los atacó, si no en contra del militarismo y el uso inadecuado del poder nuclear. Únete a esta conquista sostenible como el estratega que guiará al pueblo de Aeloria a resurgir de las cenizas, cumple misiones relacionadas a los ODS, construye edificios para sostener a tu ejército, comanda tus tropas lideradas por comandantes con habilidades que potencian a tus tropas. Vence a la corrupción existente en Grimhold y logra un futuro en el que el mundo se una para ponerle fin a las guerras de una vez por todas.

II.2 Planificación

Este epígrafe se enfoca en la organización y gestión del proyecto, siguiendo la metodología Extreme Programming (XP). La planificación en XP se centra en la flexibilidad y la colaboración continua con el cliente, utilizando iteraciones cortas para adaptarse a cambios y priorizar funcionalidades. Aquí se detallan los procesos clave que guian el desarrollo de Sustainable Conquest, desde la obtención de requisitos hasta definición de historias de usuario, la estimación de tiempo y el plan de iteraciones.

Como parte inicial de esta planificación se realizaron sesiones de tormenta de ideas (*brainstorming*) para enfocar el proyecto en una idea general y posteriormente reuniones que resultaron en la definición de los requisitos funcionales, no funcionales e historias de usuario.

II.2.1 Requisitos funcionales

Los requisitos funcionales son descripciones detalladas de las funciones, comportamientos y procesos que un sistema de software debe cumplir para satisfacer las necesidades de sus usuarios y alcanzar los objetivos del negocio. En esencia, definen “lo que el sistema debe hacer” al especificar, de forma clara y verificable, las entradas que recibe, las operaciones que realiza y las salidas que produce. Según Pressman y Maxim (2021), esta definición es esencial no solo para orientar las fases de análisis, diseño, implementación y pruebas sino también para asegurar una comunicación efectiva entre todos los involucrados en el proyecto, evitando ambigüedades y facilitando la gestión de cambios durante el desarrollo.

No	*Nombre*	*Descripción*	*Prioridad*	*Complejidad*
1.	Selección exclusiva	El sistema debe permitir al usuario seleccionar una unidad haciéndole clic izquierdo a la misma.	Alta	Media
2.	Selección múltiple	El sistema debe permitir al usuario seleccionar múltiples unidades con un cuadro de selección.	Alta	Media
3.	Selección modificable	El sistema debe permitir agregar o eliminar unidades de su selección actual.	Baja	Baja
4.	Navegación dinámica	El sistema debe poseer navegación dinámica que permita el movimiento inteligente de las unidades por el mapa.	Alta	Alta
5.	Orden de movimiento	El sistema debe permitir al usuario ordenar las unidades seleccionadas a moverse a una posición.	Alta	Media

6.	Movimiento en formación	El sistema debe permitir que las unidades se muevan en formación.	Alta	Alta
7.	Orden de alto	El sistema debe permitir al usuario ordenar a sus unidades a detenerse.	Baja	Baja
8.	Orden de ataque	El sistema debe permitir al usuario ordenar a sus unidades a atacar a una unidad.	Alta	Alta
9.	Orden de atacar-mover	El sistema debe permitir al usuario ordenar a sus unidades a atacar-moverse a una posición.	Alta	Media
10.	Manipular Cámara RTS	El sistema debe permitir al usuario manipular la cámara con las flechas o llevándo el ratón hacia los bordes de la pantalla.	Alta	Media
11.	Visualizar estado de unidad	El sistema debe permitir al usuario ver el estado de una unidad.	Alta	Media
12.	Asignar grupos de control	El sistema debe permitir al usuario asginar unidades a un grupo de control.	Alta	Media
13.	Centrar cámara en unidad	El sistema debe permitir al usuario centrar la cámara en una unidad.	Alta	Baja
14.	Mostrar unidades en minimapa	El sistema debe mostrar las unidades visibles en el minimapa con diferentes colores dependiendo de su jugador.	Media	Media
15.	Usar habilidades	El sistema debe permitir al usuario usar habilidades con la unidad seleccionada y con teclas de acceso rápido.	Media	Media
16.	Mostrar descripción de habilidades	El sistema debe mostrar la descripción de las habilidades al mover el ratón sobre los botones para activarlas.	Alta	Baja
17.	Asignar facciones	El sistema debe permitir la asignación de facciones y relaciones entre las mismas.	Alta	Alta
18.	Generar niebla de guerra	El sistema debe generar niebla de guerra en zonas no exploradas por el jugador	Alta	Alta
19.	Construcción de edificios	El sistema debe permitir al jugador la construcción de edificios	Alta	Media

20.	Entrenamiento de unidades	El sistema debe permitir al jugador el entrenamiento de unidades	Alta	Media
21.	Utilizar recursos	El sistema debe permitir al jugador la utilización de recursos para la construcción y el entrenamiento de unidades	Alta	Baja

Tabla 2: Requisitos funcionales [Elaboración propia].

II.2.2 Requisitos no funcionales

Los requisitos no funcionales son aquellos que especifican criterios que pueden usarse para juzgar la operación de un sistema, en lugar de sus comportamientos específicos. Estos requisitos se centran en aspectos como el rendimiento, la seguridad, la usabilidad y la fiabilidad del sistema, y son esenciales para garantizar que el software cumpla con las expectativas de calidad y satisfacción del usuario final (Visure Solutions, 2024).

Nombre	Tipo	Descripción
Tutorial inicial	Usabilidad	Al presentarse una funcionalidad nueva el sistema debe mostrar un tutorial explicándola.
Interfaz informativa	Usabilidad	La interfaz gráfica debe ayudar al jugador a entender el campo de batalla y saber que unidades tiene seleccionadas
Disponibilidad para Windows	Compatibilidad	El videojuego debe ser compatible con Windows 10 y versiones posteriores.
Documentación interna	Mantenibilidad	El código debe generar documentación interna accesible para futura referencia

Tabla 3: Requisitos no funcionales [Elaboración propia].

II.2.3 Historias de usuario

Las historias de usuario son una técnica fundamental en metodologías ágiles, como *Extreme Programming* (XP), son utilizadas para capturar requisitos desde la perspectiva del usuario final. Según Cohn (2020), una historia de usuario es "una descripción breve y simple de una funcionalidad deseada, expresada en lenguaje cotidiano y centrada en el valor que aporta al usuario" (p. 89). A diferencia de los documentos de requisitos tradicionales, las historias de usuario son más flexibles pero siguen un formato estandarizado:

"Como [rol], quiero [acción], para [beneficio]"

Donde el rol se refiere a la persona, la acción se refiere al requisito que se le pide al sistema y el beneficio viene dado desde la perspectiva de la persona, aunque no sea algo tangible o referente a alguna funcionalidad.

Historia de Usuario	
Numero: 1	**Nombre:** Selección exclusiva
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 1.0	**Iteración asignada: 1**
Descripción: Como jugador quiero poder seleccionar una unidad para darle órdenes.	
Observaciones: ❖ Se vacía la selección del jugador y se agrega solo la unidad seleccionada. ❖ En caso de no seleccionar unidad solamente se vacía la selección del jugador. ❖ Debe ser posible realizar esta acción a través de una interfaz gráfica. ❖ La unidad seleccionada debe destacarse sobre las demás. ❖ Debe ser posible seleccionar una unidad enemiga. ❖ No se puede seleccionar una unidad invisble.	

Tabla 4: Historia de usuario 1 [Elaboración propia].

Historia de Usuario	
Numero: 2	**Nombre:** Selección múltiple
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 1.5	**Iteración asignada: 1**
Descripción: Como jugador quiero poder seleccionar varias unidades a la vez para darles órdenes.	
Observaciones: ❖ No se pueden seleccionar unidades que no sean del jugador. ❖ Las unidades seleccionadas deben destacarse sobre las demás. ❖ En caso de no seleccionar unidad solamente se vacía la selección del jugador.	

Tabla 5: Historia de usuario 2 [Elaboración propia].

Historia de Usuario

Numero: 3	**Nombre:** Selección modificable
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 0.5	**Iteración asignada: 1**
Descripción: Como jugador quiero poder agregar o eliminar unidades de la selección para tener mejor control sobre mis unidades.	
Observaciones: ❖ No se puede modificar la selección de unidades que no sean de jugador. ❖ Si no se modifica la selección de una unidad, la selección permanece igual. ❖ Al modificar la selección de una unidad seleccionada se deselecciona. ❖ Al modificar la selección de una unidad deseleccionada se selecciona.	

Tabla 6: Historia de usuario 3 [Elaboración propia].

Historia de Usuario	
Numero: 4	**Nombre:** Navegación dinámica
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 10.0	**Iteración asignada: 1**
Descripción: Como jugador quiero que mis unidades se muevan inteligentemente para que tomen siempre el mejor camino.	
Observaciones: ❖ Las unidades deben tener en cuenta obstáculos móviles ❖ Debe actualizarse la navegación constantemente ❖ El algoritmo utilizado debe tener un rendimiento eficiente	

Tabla 7: Historia de usuario 4 [Elaboración propia].

Historia de Usuario	
Numero: 5	**Nombre:** Orden de movimiento
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 4.0	**Iteración asignada: 2**
Descripción: Como jugador quiero poder ordenar a mis unidades a moverse para poder explorar el mapa	
Observaciones: ❖ Al mover las unidades estas deben actualizar el mapa de navegación	

❖ A cada cierto intervalo de tiempo la navegación de las unidades debe actualizarse

Tabla 8: Historia de usuario 5 [Elaboración propia].

Historia de Usuario	
Numero: 6	**Nombre:** Movimiento en formación
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 5.0	**Iteración asignada: 2**
Descripción: Como jugador quiero que mis unidades se muevan en formación para que no choquen entre sí	
Observaciones: ❖ En caso de haber una sola unidad no hay formación. ❖ Las unidades se forman de forma rectangular. ❖ El tamaño de las filas y columnas depende de la cantidad de unidades. ❖ Las formaciones se rotan dependiendo de la posición de inicio de las unidades seleccionadas.	

Tabla 9: Historia de usuario 6 [Elaboración propia].

Historia de Usuario	
Numero: 7	**Nombre:** Orden de alto
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 1.0	**Iteración asignada: 2**
Descripción: Como jugador quiero poder ordenar mis unidades a detenerse para mayor control táctico	
Observaciones: ❖ Si la unidad se encuentra en movimiento se detiene ❖ Si la unidad se encuentra realizando alguna acción la cancela ❖ Debe ser posible detener varias unidades	

Tabla 10: Historia de usuario 7 [Elaboración propia].

Historia de Usuario	
Numero: 8	**Nombre:** Orden de ataque
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 3.0	**Iteración asignada: 2**

Descripción: Como jugador quiero poder atacar unidades para poder combatir enemigos.
Observaciones: ❖ Debe ser posible atacar cualquier unidad si se usa la tecla de acceso rápido ❖ Debe ejecutarse al hacer clic derecho sobre una unidad enemiga con unidades seleccionadas

Tabla 11: Historia de usuario 8 [Elaboración propia].

Historia de Usuario	
Numero: 9	**Nombre:** Orden de atacar-mover
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 2.0	**Iteración asignada: 2**
Descripción: Como jugador quiero poder dar la orden de atacar-mover para que mis unidades avancen atacando enemigos en el camino.	
Observaciones: ❖ Si no se detecta un enemigo la unidad se mueve ❖ Si se detecta un enemigo se ataca ❖ Al derrotar un enemigo la unidad sigue en su camino nuevamente.	

Tabla 12: Historia de usuario 9 [Elaboración propia].

Historia de Usuario	
Numero: 10	**Nombre:** Manipular cámara RTS
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 1.0	**Iteración asignada: 3**
Descripción: Como jugador quiero poder mover la cámara por el mapa para observar y explorar el mapa.	
Observaciones: ❖ La cámara debe poder manipularse por las flechas. ❖ La cámara debe moverse al llevar el ratón a las esquinas de la pantalla. ❖ La cámara no debe salir de los límites del mapa.	

Tabla 13: Historia de usuario 10 [Elaboración propia].

Historia de Usuario

Numero: 11	**Nombre:** Visualizar estado de unidad
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 1.5	**Iteración asignada: 3**
Descripción: Como jugador quiero poder ver el estado y los atributos de mis unidades para poder tomar decisiones precisas.	
Observaciones: ❖ Se visualiza el estado de la unidad principal seleccionada. ❖ La unidad principal debe poder cambiarse por la interfaz. ❖ Las unidades de otros jugadores también pueden ser visualizadas.	

Tabla 14: Historia de usuario 11 [Elaboración propia].

Historia de Usuario	
Numero: 12	**Nombre:** Asignar grupos de control
Prioridad: Alta	**Riesgo: Bajo**
Puntos de estimación: 1.2	**Iteración asignada: 3**
Descripción: Como jugador quiero poder asignar grupos de control para seleccionar unidades de forma rápida y eficiente.	
Observaciones: ❖ Solo unidades del jugador se añaden al grupo de control	

Tabla 15: Historia de usuario 12 [Elaboración propia].

Historia de Usuario	
Numero: 13	**Nombre:** Centrar cámara en unidad
Prioridad: Baja	**Riesgo: Bajo**
Puntos de estimación: 0.5	**Iteración asignada: 3**
Descripción: Como jugador quiero poder centrar la cámara en una unidad para ver su posición rápidamente en el campo de batalla.	
Observaciones: ❖ Debe poder ejecutarse mediante la interfaz gráfica.	

Tabla 16: Historia de usuario 13 [Elaboración propia].

Historia de Usuario	
Numero: 14	**Nombre:** Mostrar unidades en minimapa
Prioridad: Alta	**Riesgo: Medio**
Puntos de estimación: 8.0	**Iteración asignada: 3**
Descripción: Como jugador quiero poder visualizar las unidades en el minimapa para poder tener una idea general del campo de batalla.	
Observaciones: ❖ Debe mostrar todas las unidades visibles por el jugador. ❖ Debe esconder las unidades que se hagan invisibles. ❖ Debe mostrar las unidades con diferentes colores dependiendo del jugador. ❖ Debe actualizarse constantemente.	

Tabla 17: Historia de usuario 14 [Elaboración propia].

Historia de Usuario	
Numero: 15	**Nombre:** Usar habilidades
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 6.0	**Iteración asignada: 4**
Descripción: Como jugador quiero poder usar habilidades para tener una ventaja táctica	
Observaciones: ❖ Existen habilidades pasivas que funcionan sin tener que activarse. ❖ Existen habilidades de objetivo que requieren marcar un objetivo en que usarlas. ❖ Existen habilidades sin objetivo que se utilizan en el lugar. ❖ Al utilizar habilidades se interrumpe la acción de la unidad, con algunas excepciones. ❖ Las habilidades tienen tiempo de recarga. ❖ Deben poder usarse por teclas rápidas.	

Tabla 18: Historia de usuario 15 [Elaboración propia].

Historia de Usuario	
Numero: 16	**Nombre:** Mostrar descripción de habilidades
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 1.0	**Iteración asignada: 4**

Descripción: Como jugador quiero poder ver la descripción de las habilidades para entender su uso.
Observaciones: ❖ Se ejecuta al mover el ratón sobre la habilidad ❖ Debe mostrar el nombre de la habilidad ❖ Debe mostrar su nivel si tiene varios ❖ Debe mostrar la explicación sobre como funciona la habilidad ❖ Si tiene costo de recursos debe reflejarse

Tabla 19: Historia de usuario 16 [Elaboración propia].

Historia de Usuario	
Numero: 17	**Nombre:** Asignar facciones
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 4.0	**Iteración asignada: 4**
Descripción: Como jugador quiero que se asignen facciones y relaciones entre ellas para una mejor jugabilidad	
Observaciones: ❖ Las relaciones existentes son Misma facción, Amigo, Neutral y Hostil. ❖ La relación por defecto si no se asigna es Hostil. ❖ Una facción consigo misma siempre va a tener la relación Misma facción.	

Tabla 20: Historia de usuario 17 [Elaboración propia].

Historia de Usuario	
Numero: 18	**Nombre:** Niebla de guerra
Prioridad: Media	**Riesgo: Alto**
Puntos de estimación: 20.0	**Iteración asignada: 5**
Descripción: Como jugador quiero que exista un efecto de niebla de guerra para poder emplear tácticas que dependan de la visibilidad.	

Observaciones: ❖ Las unidades del jugador disuelven la niebla según su radio de visibilidad. ❖ La niebla debe actualizarse sin disminuir visiblemente el rendimiento del videojuego. ❖ Las unidades dentro de la niebla deben hacerse invisibles y fuera de ella visibles.

Tabla 21: Historia de usuario 18 [Elaboración propia].

Historia de Usuario	
Numero: 19	**Nombre:** Construcción de edificios
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 5.0	**Iteración asignada: 6**
Descripción: Como jugador quiero poder construir edificios para poder crear una base.	
Observaciones: ❖ Construir edificio es una habilidad que no interrumpe al lanzador. ❖ Debe tener una cantidad suficiente de recursos para ejecutarla. ❖ Debe verse una previsualización del edificio. ❖ Al hacer clic izquierdo en un lugar válido se construye el edificio. ❖ Si el lugar elegido no es válido debe mostrarse un mensaje de error. ❖ Si el lugar elegido es válido pero no está en el rango de la habilidad el lanzador debe acercarse al punto. ❖ Debe ser posible poner varios edificios en cola. ❖ Presionando clic derecho se cancela la construcción del edificio actual. ❖ Presionando escape se cancelan todas las construcciones en cola.	

Tabla 22: Historia de usuario 19 [Elaboración propia].

Historia de Usuario	
Numero: 20	**Nombre:** Entrenamiento de unidades
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 7.0	**Iteración asignada: 6**
Descripción: Como jugador quiero poder entrenar unidades para abastecer mi ejército de tropas	

Observaciones:
❖ Entrenar unidad es una habilidad que no interrumpe al lanzador. ❖ Debe tener una cantidad suficiente de recursos para ejecutarla. ❖ De no tener suficientes recursos se debe mostrar un mensaje de error ❖ De estar seleccionada solamente la unidad que está entrenando se muestra en la interfaz la cola de entrenamiento ❖ La cola de entrenamiento tiene un límite ❖ Al intentar pasarse del límite de la cola de entrenamiento se muestra un mensaje de error ❖ Cada unidad entrenada tiene su tiempo de entrenamiento ❖ Al terminar de entrenar una unidad debe aparecer en el espacio válido más cercano

Tabla 23: Historia de usuario 20 [Elaboración propia].

Historia de Usuario	
Numero: 21	**Nombre:** Utilizar recursos
Prioridad: Alta	**Riesgo: Alto**
Puntos de estimación: 4.0	**Iteración asignada: 6**
Descripción: Como jugador quiero poder utilizar los recursos para entrenar unidades, construir edificios efectivamente	
Observaciones: ❖ Existen edificios con habilidades para producir estos recursos. ❖ Los recursos se gastan para entrenar o construir. ❖ Los recursos son créditos, provisiones y electricidad. ❖ Los créditos se gastan en todo y se producen a cada segundo. ❖ Las provisiones se gastan en las unidades. ❖ La electricidad se gasta para construir varios edificios.	

Tabla 24: Historia de usuario 21 [Elaboración propia].

II.2.4 Estimación de tiempo de desarrollo

Los puntos de estimación en el desarrollo ágil se utilizan para medir el esfuerzo relativo requerido para completar una historia de usuario en lugar de estimar el tiempo exacto, lo que sirve para evaluar su complejidad, comparar historias de usuario y planificar las iteraciones.

No.	Historias de Usuario	Puntos de Estimación (Días)
1	Selección exclusiva	1.0
2	Selección múltiple	1.5
3	Selección modificable	0.5
4	Navegación dinámica	10.0
5	Orden de movimiento	4.0
6	Movimiento en formación	5.0
7	Orden de alto	1.0
8	Orden de ataque	3.0
9	Orden de atacar-mover	2.0
10	Manipular Cámara RTS	1.0
11	Visualizar estado de unidad	1.5
12	Asignar grupos de control	1.2
13	Centrar cámara en unidad	0.5
14	Mostrar unidades en minimapa	8.0
15	Usar habilidades	6.0
16	Mostrar descripción de habilidades	1.0
17	Asignar facciones	4.0
18	Generar niebla de guerra	20.0
19	Construcción de edificios	5.0
20	Entrenamiento de unidades	7.0
21	Utilizar recursos	4.0

Tabla 25: Puntos de estimación [Elaboración propia].

II.2.5 Plan de iteraciones

El plan de iteraciones es una herramienta esencial para organizar y coordinar el trabajo del equipo de desarrollo. Cada iteración, que suele durar entre una y tres semanas, comienza con

una reunión de planificación en la que el cliente presenta las historias de usuario priorizadas, y el equipo de desarrollo estima y se compromete a completar las tareas necesarias para implementar esas historias (Paasivaara & Kruchten, 2020). Este enfoque permite entregar incrementos funcionales del software de manera continua, asegurando que el producto evolucione de acuerdo con las necesidades cambiantes del cliente (Paasivaara & Kruchten, 2020). La planificación de iteraciones también incluye la asignación de tareas específicas a los desarrolladores, la estimación de su duración y la identificación de dependencias, lo que facilita una ejecución eficiente y enfocada en los objetivos del cliente (Paasivaara & Kruchten, 2020).

Iteración	Historia de usuario		Duración
1	1	Selección exclusiva	13.0
	2	Selección múltiple	
	3	Selección modificable	
	4	Navegación dinámica	
2	5	Orden de movimiento	15.0
	6	Movimiento en formación	
	7	Orden de alto	
	8	Orden de ataque	
	9	Orden de atacar-mover	
3	10	Manipular Cámara RTS	12.2
	11	Visualizar estado de unidad	
	12	Asignar grupos de control	
	13	Centrar cámara en unidad	
	14	Mostrar unidades en minimapa	
4	15	Usar habilidades	11.0
	16	Mostrar descripción de habilidades	
	17	Asignar facciones	

5	18	Generar niebla de guerra	20.0
6	19	Construcción de edificios	16.0
	20	Entrenamiento de unidades	
	21	Utilizar recursos	

Tabla 26*: Plan de iteraciones [Elaboración propia].*

II.3 Diseño

Este epígrafe constituye la transición esencial entre la planificación teórica y la implementación práctica, donde se materializan los requisitos en soluciones técnicas concretas. Este apartado se centra en tres pilares fundamentales: (1) la arquitectura del sistema, detallando cómo la estructura basada en nodos y el *SceneTree* de Godot permiten organizar los componentes clave del videojuego, (2) los prototipos de interfaces gráficas y (3) las tarjetas CRC (Class-Responsibility-Collaboration) que definen la interacción entre clases.

II.3.1 Arquitectura utilizada

Godot utiliza una arquitectura jerárquica basada en nodos, donde cada elemento del juego (desde un personaje hasta un sonido) es un nodo (objeto) que hereda de la clase *Node*. Según la documentación oficial (Godot Engine, 2023), los nodos son "bloques de construcción modulares que se combinan para formar escenas complejas". El *SceneTree* (árbol de escenas) es la estructura que gestiona todas las escenas y nodos en tiempo de ejecución. Funciona como un árbol de procesamiento que organiza la orden de ejecución de métodos, la jerarquia de transformaciones para establecer las posiciones de los nodos en la pantalla y la gestión de señales que comunican a los nodos entre sí.

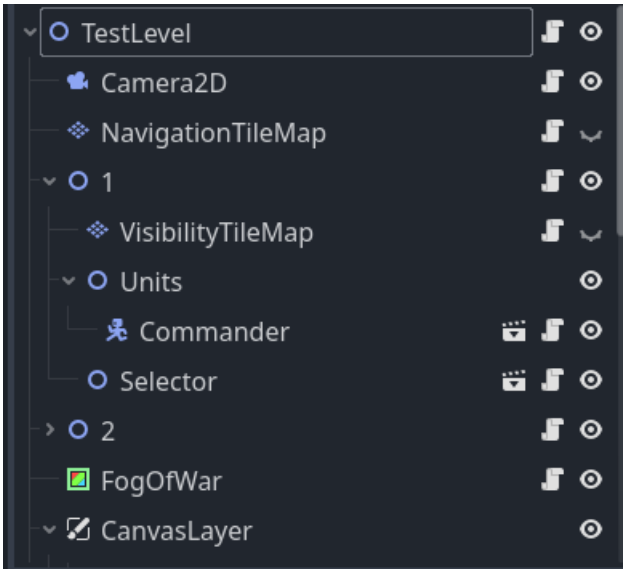

Figura 1: SceneTree de Godot [Elaboración propia].

El uso del *SceneTree* trae consigo ventajas como el aumento de la modularidad, permitiendo tener una escena para cada funcionalidad específica, como podría ser una unidad del jugador e instanciar esta dentro de otra escena más grande que sería el nivel. De esta manera un nivel puede tener varias unidades, cada una con sus propiedades. Pero al momento de necesitar cambiarlas todas se pueden realizar cambios en la escena de la unidad, reflejándose en todas las unidades. Otra de sus ventajas es el rendimiento que mejora debido a que evita las actualizaciones innecesarias y permite operaciones masivas como llamadas de métodos en grupos. Su diseño trae un enfoque de composición sobre herencia para no depender de jerarquías rígidas de clases y posee funcionalidades para realizar cambios de escenas. El *SceneTree* aparte de ser la arquitectura por defecto de Godot es excelente para Sustainable Conquest porque facilita la instanciación masiva y gestión dinámica de unidades durante la partida. La comunicación entre estos elementos se simplifica gracias al sistema de señales integrado, permitiendo que eventos como la gestión de recursos o los combates se coordinen eficientemente sin acoplamiento rígido entre componentes. Además, el *SceneTree* optimiza automáticamente el rendimiento al gestionar el orden de procesamiento y renderizado, algo crítico en juegos RTS donde cientos de entidades pueden interactuar simultáneamente en pantalla. La capacidad de pausar o manipular selectivamente ramas enteras del árbol de escenas es particularmente útil para implementar mecánicas como pausas tácticas o modos de construcción.

II.3.3 Prototipos de interfaces gráficas

En este epígrafe se presentan las principales interfaces gráficas en su forma de prototipo, describiendo además cada uno de sus elementos y funcionalidades asociadas.

La barra de estado describe las propiedades que tiene la unidad seleccionada, primeramente el retrato de la unidad, seguido de dos barras con la salud y energía de la unidad respectivamente, debajo de estas vienen los atributos como ataque, defensa, poder mágico, resistencia a la magia, velocidad de movimiento y velocidad de ataque. Además enseña si la unidad sufre de algún estado alterado tanto positivo como negativo y en caso de ser un comandante muestra su nivel actual debajo del retrato y los puntos de habilidad que posee.

Figura 2: Barra de estado [Elaboración propia].

Figura 3: Barra de estado para Comandantes [Elaboración propia].

El contenedor de unidades muestra las unidades seleccionadas actualmente además de las guardadas en un grupo de control. Este trae un panel para las seleccionadas y para grupos de control del uno al cero (1,2,3,4,5,6,7,8,9,0) mostrando el ícono de cada unidad con su barra de salud. Esta permite agregar o eliminar unidades de la selección, seleccionar una en específico o centrar la cámara en alguna de ellas.

Figura 4: Contenedor de unidades [Elaboración propia].

La barra de habilidades muestra las habilidades de la unidad seleccionada. El botón superior de cada habilidad se ilumina cuando existe la cantidad suficiente de puntos de habilidad para subirla de nivel y se sombrea cuando está en nivel máximo o no existe esta cantidad, el mismo no se muestra en caso de la habilidad no tener varios niveles. El botón inferior es el botón que activa la habilidad, al poner el ratón encima de estos sale un cartel flotante con el nombre, nivel si tiene y la descripción de la habilidad. Se sombrea un botón cuando la habilidad no se puede utilizar. Además del clic izquierdo se pueden usar las teclas Q, W, E, R, T, Y para lanzarlos por tecla rápida

Figura 5: Barra de habilidades [Elaboración propia].

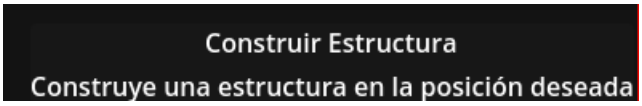

Figura 6: Descripción de habilidad [Elaboración propia].

El contenedor de recursos muestra primeramente los créditos que son la moneda virtual del videojuego para producir unidades o construir edificios, representados por una tarjeta de crédito. Luego, representadas por un contenedor se tienen las provisiones que dictan el sustento o la cantidad de unidades que se tienen y la cantidad máxima, cabe destacar que algunas unidades usan más de un punto de provisiones. Simbolizada por un rayo se tiene la electricidad, recurso necesario similar a las provisiones pero para sustentar los edficios que se construyan.

Figura 7: Contenedor de recursos [Elaboración propia].

El minimapa se encarga de proporcionar una vista del terreno completo de una forma más sencilla y accesible para el jugador, permite centrar la cámara en la posición en que haga clic dentro del mismo y posee representación en colores de las unidades del jugador, amigas y enemigas dentro del rango de visión.

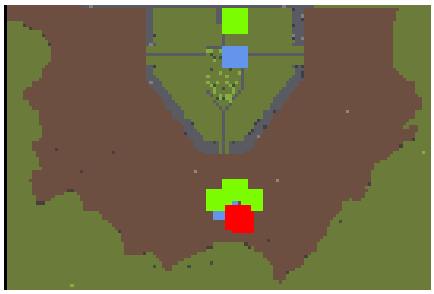

Figura 8: Minimapa [Elaboración propia].

La cola de entrenamiento muestra las unidades que se están entrenando en orden y su progreso. Se pueden quitar unidades de la misma haciendo clic izquierdo sobre el ícono de la unidad, además al mover el ratón sobre las unidades se muestra la descripción de las mismas

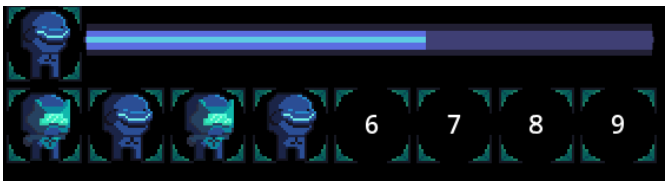

Figura 9: Cola de entrenamiento [Elaboración propia].

El menú de pausa está compuesto por 4 botones, el de resumir para seguir la partida, el de ayuda para ver los tutoriales disponibles, el de salir para terminar el juego y el de pantalla completa para activar o desactivar esta opción.

Figura 10: Menú de pausa [Elaboración propia].

II.3.3 Tarjetas CRC

Las tarjetas CRC (*Class-Responsibility-Collaboration*) constituyen una técnica fundamental en el diseño ágil contemporáneo, particularmente relevante en metodologías como *Extreme Programming* (XP). Son una herramienta efectiva para la conceptualización y diseño de sistemas orientados a objetos, que facilita la identificación de las clases, sus responsabilidades y las colaboraciones necesarias entre ellas. Estas tarjetas se utilizan en equipos ágiles para fomentar la comunicación colaborativa durante la fase de diseño, permitiendo que los desarrolladores comprendan mejor las interacciones dentro del sistema (Sommerville, 2021). El formato típico de las tarjetas CRC mantiene su estructura tripartita esencial (clase, responsabilidad, colaboración), pero ha evolucionado para incorporar elementos visuales adicionales como colores y símbolos que representan patrones de diseño frecuentes en el desarrollo moderno de software..

Clase	Responsabilidad	Colaborador
Unit	- navegar por el terreno - mostrar animaciones - usar habilidades - colisionar con el entorno - atacar unidades	Skill Stats Selector NavigationTileMap

Tabla 27: Tarjeta CRC Unit [Elaboración propia].

Clase	Responsabilidad	Colaborador
Selector	- seleccionar unidades según la posición del ratón - mostrar rectángulo traslúcido al arrastrar el ratón - enviar órdenes a las unidades seleccionadas - indicar la posición en la formación que ocupa cada unidad	Unit FormationManager

Tabla 28: Tarjeta CRC Selector [Elaboración propia].

Clase	Responsabilidad	Colaborador
Map	- establecer relaciones entre facciones - conectar interfaz con el jugador	FactionManager Unit Player

	- registrar unidades existentes por ID y facción en tiempo de ejecución - registrar unidades existentes en la navegación	IDGenerator NavigationTileMap

Tabla 29: Tarjeta CRC Map [Elaboración propia].

Clase	Responsabilidad	Colaborador
RTSCamera	- mover la camara por las flechas direccionales - mover la cámara al posicionar el ratón en los bordes de la pantalla - centrar cámara en una posición	

Tabla 30: Tarjeta CRC RTSCamera [Elaboración propia].

Clase	Responsabilidad	Colaborador
IDGenerator	- generar ID autoincremental	

Tabla 31: Tarjeta CRC IDGenerator [Elaboración propia].

Clase	Responsabilidad	Colaborador
FormationManager	- crear formación - asignar posición de formación según cercanía	

Tabla 32: Tarjeta CRC FormationManager [Elaboración propia].

Clase	Responsabilidad	Colaborador
FactionManager	- crear facciones - establecer relaciones entre facciones - obtener relación entre dos facciones	

Tabla 33: Tarjeta CRC FactionManager [Elaboración propia].

Clase	Responsabilidad	Colaborador
TargetManager	- devolver tipo de objetivo dada la unidad que ejecuta la acción y su unidad objetivo (misma unidad, misma facción, amiga, neutral, hostil)	FactionManager

Tabla 34: Tarjeta CRC TargetManager [Elaboración propia].

Clase	Responsabilidad	Colaborador

FogOfWar	- disolver efecto de niebla de guerra dada una FogInfo	FogInfo

Tabla 35: Tarjeta CRC FogOfWar [Elaboración propia].

Clase	Responsabilidad	Colaborador
FogInfo	- guardar parámetros para disolver la niebla en las posiciones correspondientes	

Tabla 36: Tarjeta CRC FogInfo [Elaboración propia].

Clase	Responsabilidad	Colaborador
Minimap	- mostrar posiciones de unidades visibles (dentro del VisibilityTileMap) - recoger datos para rellenar la FogInfo - enviar la FogInfo a la FogOfWar - actualizar a cada cierto intervalo de tiempo	VisibilityTileMap FogOfWar FogInfo Unit

Tabla 37: Tarjeta CRC Minimap [Elaboración propia].

Clase	Responsabilidad	Colaborador
Stats	- guardar parámetros que difieren entre unidades (salud, energía, ataque, defensa, magia, resistencia)	

Tabla 38: Tarjeta CRC Stats [Elaboración propia].

Clase	Responsabilidad	Colaborador
StatsPerLevel	- guardar cantidad de puntos de cada parámetro que aumentan por cada nivel.	

Tabla 39: Tarjeta CRC StatsPerLevel [Elaboración propia].

Clase	Responsabilidad	Colaborador
Commander	- obtener experiencia por enemigos derrotados - subir habilidades de nivel - aumentar parámetros al subir de nivel	Unit Stats StatsPerLevel

Tabla 40: Tarjeta CRC Commander [Elaboración propia].

Clase	Responsabilidad	Colaborador
Player	- gestionar cantidades de recursos (dinero, comida y electricidad)	

Tabla 41: Tarjeta CRC Player [Elaboración propia].

Clase	Responsabilidad	Colaborador
VisibilityTileMap	- guardar casillas de terreno visible	

Tabla 42: Tarjeta CRC VisibilityTileMap [Elaboración propia].

Clase	Responsabilidad	Colaborador
AIComponent	- guardar estado anterior y actual - manejar lógica de cambio de estado - actualizar estado actual	State StateManager

Tabla 43: Tarjeta CRC AIComponent [Elaboración propia].

Clase	Responsabilidad	Colaborador
State	- guardar funcionalidades al entrar, actualizar o salir del estado	

Tabla 44: Tarjeta CRC State [Elaboración propia].

Clase	Responsabilidad	Colaborador
Skill	- guardar el nivel, un booleano si es visible o no y la SkillData - determinar si está en enfríamiento (cooldown) - devolver el tiempo de enfríamiento restante - guardar funcionalidades para cuando se agrega, se usa o se remueve la habilidad	SkillData

Tabla 45: Tarjeta CRC Skill [Elaboración propia].

Clase	Responsabilidad	Colaborador
SkillData	- guardar datos de la habilidad (nombre, descripción, tiempo de enfríamiento, entre otros)	

Tabla 46: Tarjeta CRC SkillData [Elaboración propia].

Clase	Responsabilidad	Colaborador
StateManager	- proveer acceso a todas las máquinas de estado de forma global	StateMachine

Tabla 47: Tarjeta CRC StateManager [Elaboración propia].

Clase	Responsabilidad	Colaborador
StateMachine	- guardar estados según el tipo de máquina de estados	State

	- proveer el comportamiento deseado a cada estado	

Tabla 48: Tarjeta CRC StateMachine [Elaboración propia].

II.4 Implementación

Este epígrafe describe los aspectos necesarios a tener en cuenta a la hora de implementar el videojuego. Se abordan aspectos clave como los estándares y convenciones de codificación, que aseguran la coherencia, legibilidad y mantenibilidad del código, así como el uso de patrones de diseño, que proporcionan soluciones reutilizables para problemas comunes y guían la estructuración del software y las tareas de ingeniería. En conjunto, se resaltan las prácticas adoptadas para garantizar un código de alta calidad, flexible, escalable y fácil de mantener durante el ciclo de vida del proyecto.

II.4.1 Estándares de codificación

Los estándares de codificación son conjuntos de reglas formales y documentadas que definen cómo debe escribirse, organizarse y estructurarse el código fuente en un proyecto de software. Los estándares de codificación y las métricas asociadas a la legibilidad actúan como guías fundamentales para asegurar la coherencia, legibilidad y mantenibilidad del código, sin importar cuál sea el desarrollador que lo escriba. La correcta aplicación de dichos estándares permite que el software sea más comprensible y, por consiguiente, facilite las tareas de mantenimiento y evolución continua, reduciendo costos y esfuerzos a lo largo del ciclo de vida del sistema. En este sentido, como señalan Echavarría-Flórez y Restrepo-Calle (2020) en su revisión sistemática de métricas de legibilidad, una alta claridad en el código incide positivamente en la eficiencia del mantenimiento, estableciendo una base para el desarrollo de software de calidad. Incluyen directivas sobre nomenclatura (ej: *camelCase* para variables, *PascalCase* para clases), indentación (espacios vs. tabulaciones), longitud máxima de líneas, estructura de archivos y comentarios.

En el desarrollo de Sustainable conquest se utilizan los estándares de GDScript tales como uso de tabulación como indentación, PascalCase para el nombre de clases y comentarios de documentación en todas las propiedades exportables. Además el equipo propuso como convenciones el uso de camelCase para nombre de variables y funciones y aplicación obligatoria del tipado estático excepto en casos excepcionales. En cuanto a la estructura de

archivos se ejecuta una organización jerárquica en vez de relacional debido al tamaño medianamente grande del videojuego

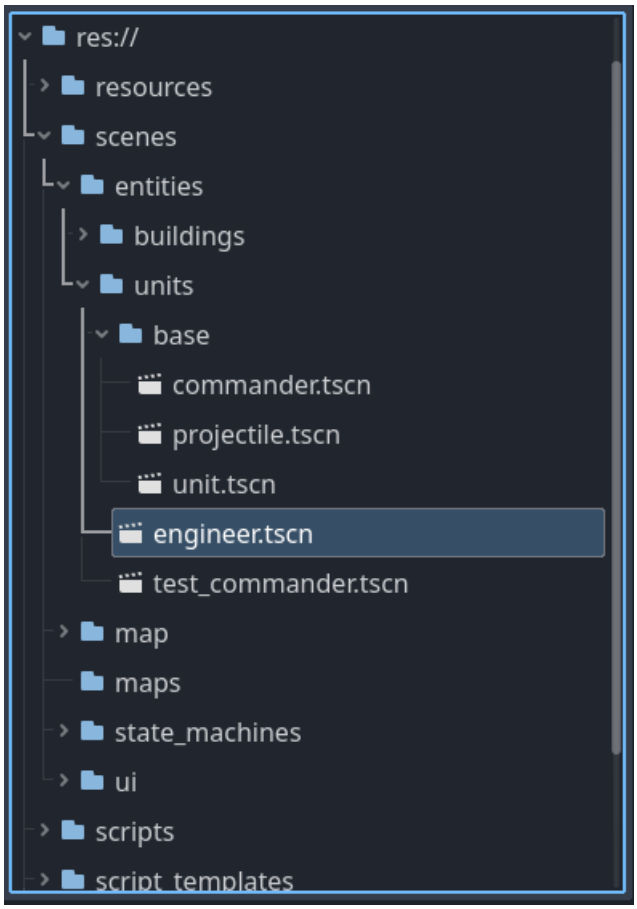

Figura 11: Estructura de archivos jerárquica [Elaboración propia].

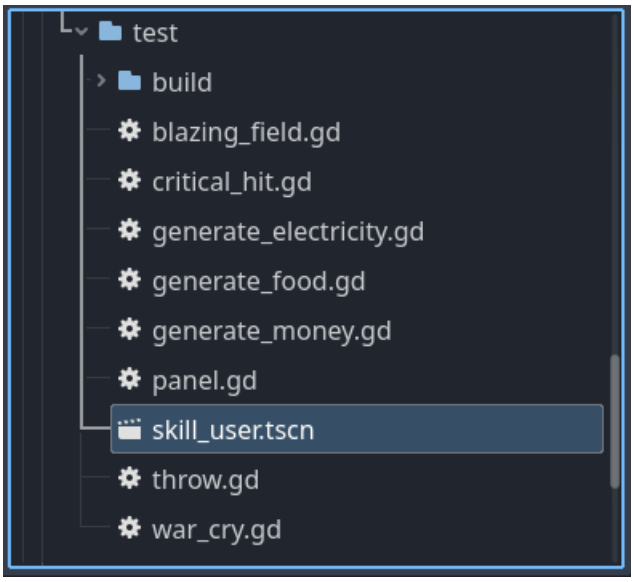

Figura 12: Estructura de archivos relacional [Elaboración propia].

```
92  func isNavigationFinished() -> bool:
93      return pathIndex == -1 or pathIndex >= path.size()
94
95
96  func navigate() -> void:
97      moveDirection = global_position.direction_to(path[pathIndex])
98      move(moveDirection * currentStats.movementSpeed)
99      if global_position.distance_squared_to(path[pathIndex]) < arrivalDistance:
100         pathIndex += 1
```

Figura 13: Uso de identaciones en GDScript [Elaboración propia].

```
1   ##Holds and control the resources the player has
2   class_name Player extends Node2D
3
4   ##Emits when the money property is changed
5   signal moneyChanged(newMoney : int)
6   ##Emits when the food or foodMax properties are changed
7   signal foodChanged(newFood : int, newFoodMax : int)
8   ##Emits when the electricityUsed or electricityMax properties are changed
9   signal electricityChanged(newElectricity : int, newElectricityMax : int)
10
11  @onready var selector = get_node_or_null("Selector")
12  @onready var unitsLayer = $Units
13
14  ##Property that tells how much money the player has
15  @export var money : int = 0:
16      set(_money):
17          money =_money
18          moneyChanged.emit(money)
19
20  ##Property that tells what is the maximum ammount of food the player has
21  @export var foodMax : int = 0:
22      set(_foodMax):
23          foodMax = _foodMax
24          foodChanged.emit(food, foodMax)
```

Figura 14: Clase Player con comentarios de documentación [Elaboración propia].

Class: Player

Inherits: Node2D < CanvasItem < Node < Object

Holds and control the resources the player has

Properties

```
    int  electricityMax   [default: 0]                    [property: setter]
    int  electricityUsed  [default: 0]                    [property: setter]
    int  food             [default: 0]                    [property: setter]
    int  foodMax          [default: 0]                    [property: setter]
    int  money            [default: 0]                    [property: setter]
Variant  selector         [default: get_node_or_null(...)]
Variant  unitsLayer       [default: <unknown>]
```

Methods

```
int  getUsableElectricity()
int  getUsableFood()
int  getUsableMaxElectricity()
int  getUsableMaxFood()
```

Signals

- `electricityChanged(newElectricity: int, newElectricityMax: int)`
  Emits when the electricityUsed or electricityMax properties are changed
- `foodChanged(newFood: int, newFoodMax: int)`

Figura 15: Documentación autogenerada de clase Player [Elaboración propia].

II.4.2 Patrones utilizados

Los patrones de diseño constituyen soluciones comprobadas y reutilizables para resolver problemas comunes en la arquitectura de software. Estos patrones ofrecen una guía estructurada que define la interacción y las responsabilidades de clases y objetos, permitiendo implementar soluciones robustas y escalables. Según Vale et al. (2022), la aplicación de dichos patrones no solo mejora la organización y la modularidad del código, sino que también incide positivamente en atributos críticos de calidad, aspectos que cobran especial relevancia en entornos basados en microservicios y sistemas de alta complejidad.. Estos patrones emergieron como respuesta a la necesidad de estandarizar buenas prácticas, evitando reinventar soluciones para problemas recurrentes. los patrones de diseño no deben interpretarse como fragmentos de código listos para copiar, sino como plantillas conceptuales que orientan la estructuración del software en la búsqueda de cualidades esenciales como la flexibilidad, mantenibilidad y escalabilidad. Esta abstracción facilita la resolución de problemas comunes durante el desarrollo y permite adaptar las soluciones a distintos contextos y

requerimientos específicos del sistema (Richards & Ford, 2020). Se documentan típicamente con: (1) un nombre descriptivo, (2) el problema que resuelve, (3) la solución propuesta, y (4) las consecuencias de su implementación.

Dentro del universo de patrones, dos categorías destacan por su influencia histórica y adopción generalizada:

GRASP (General Responsibility Assignment Software Patterns): Desarrollados por Craig Larman, los principios GRASP constituyen un conjunto de pautas esenciales para la asignación de responsabilidades en sistemas orientados a objetos. Estos buscan responder de manera clara a la pregunta: "¿Qué objeto debe ser responsable de cierta funcionalidad?". La aplicación de estos principios permite alcanzar una alta cohesión y un bajo acoplamiento, lo que resulta fundamental para mejorar la mantenibilidad y escalabilidad del software (Fluent C++, 2021). A diferencia de los patrones GOF (más específicos), los GRASP son conceptos abstractos que sirven como base para decisiones de diseño. Son particularmente útiles en etapas iniciales de diseño.

En el desarrollo de la solución se utilizó el patrón Experto de información, el cual propone que la clase que posee la información necesaria para ejecutar un método es la que debe tener dicho método, como en el caso de los métodos que acceden a las celdas de navegación en el *NavigationTileMap.*

```
52  ##Returns if the space from initialPosition to finalPosition is full of available cells
53  func isSpaceUsed(initialPosition : Vector2, finalPosition : Vector2) -> bool:
54      var initialCoords : Vector2i = local_to_map(to_local(initialPosition))
55      var finalCoords : Vector2i = local_to_map(to_local(finalPosition))
56      var rows : int = ceil(abs(initialCoords.y - finalCoords.y))+1
57      var columns : int = ceil(abs(initialCoords.x - finalCoords.x))+1
58      for row : int in rows:
59          for column : int in columns:
60              var currentCoords : Vector2i = Vector2i(initialCoords.x + column, initialCoords.y + row)
61              var tileSourceID = get_cell_source_id(currentCoords)
62              if tileSourceID != 0:
63                  return false
64      return true
65
66  ##Returns the closest available cell position in global coordinates given an initial position
67  func getClosestAvailableCell(initialCell : Vector2i) -> Vector2i:
68      var cell = initialCell
69      var cells : Array[Vector2i] = [cell]
70      while not cells.is_empty():
71          cell = cells.pop_front()
72          if get_cell_source_id(cell) == 0:
73              return cell
74          cells.append_array(get_surrounding_cells(cell))
75      return initialCell
76
77  ##Returns the closest available position given an initialPosition in global coordinates
78  func getClosestAvailablePosition(initialPosition : Vector2) -> Vector2:
79      var currentCell : Vector2i = local_to_map(initialPosition)
80      return map_to_local(getClosestAvailableCell(currentCell))
```

Figura 16: Ejemplo de patrón experto en la clase NavigationTileMap [Elaboración propia].

El patrón Bajo acoplamiento se ve reflejado en varias clases como FogInfo y VisibilityTileMap que solamente traen información y la manipulan sin depender de ninguna otra clase

```
1   class_name FogInfo extends Resource
2
3   ##Ammount of UVs that will be processed
4   @export var clearCount : int = 0
5
6   ##UV coordinates of the positions to clear
7   @export var clearPositions : Array[Vector2] = []
8
9   ##Radii to clear the UVs in a circle
10  @export var visibilityRadii : Array[int] = []
11
12  ##Size of the fog texture
13  @export var fogSize : Vector2 = Vector2.ZERO
14
```

Figura 17: Ejemplo de patrón Bajo acoplamiento en clase FogInfo [Elaboración propia].

```
1   class_name VisibilityTileMap extends TileMapLayer
2
3   ##Returns wether a globalPosition is visible (inside the fog or not)
4   func isSpaceVisible(globalPosition : Vector2) -> bool:
5       var localPosition : Vector2 = to_local(globalPosition)
6       var mapPosition : Vector2i = local_to_map(localPosition)
7       var sourceID : int = get_cell_source_id(mapPosition)
8       return sourceID != -1
9
10
11  func isCoordVisible(coords : Vector2i):
12      return get_cell_source_id(coords) != -1
13
14
15  func revealArea(centerPosition : Vector2, visibilityRadius : int) -> void:
16      var visibilityVector : Vector2 = Vector2(visibilityRadius, visibilityRadius)
17      var initialPosition : Vector2 = centerPosition - visibilityVector
18      var finalPosition : Vector2 = centerPosition + visibilityVector
19      revealCells(initialPosition, finalPosition, visibilityRadius)
```

Figura 18: Ejemplo del patrón bajo acoplamiento en la clase VisibilityTileMap [Elaboración propia].

GOF (Gang of Four): documentados en el seminal libro *Design Patterns: Elements of Reusable Object-Oriented Software* (Gamma et al., 2021, ed. aniversario), se exponen 23 patrones de diseño que representan soluciones concretas a problemas recurrentes en la ingeniería de software orientado a objetos. Estos patrones se dividen en tres categorías principales. Los patrones creacionales abordan la forma de crear instancias de objetos, ejemplificados en patrones como *Singleton* y *Factory Method*. Los estructurales, centrados en la composición de clases y objetos para formar sistemas arquitectónicamente sólidos, como se aprecia en *Adapter* y *Composite*. Por último, los patrones de comportamiento definen las interacciones y la distribución de responsabilidades entre objetos, siendo *Observer* y *Strategy* ejemplos representativos.

En Godot por defecto vienen incorporadas soluciones que usan los patrones *Singleton* y *Observer*, *autoloads* y señales respectivamente. Los primeros tienen que registrarse en la pestaña *globals* dentro de los ajustes del proyecto y los segundos pueden registrarse por el panel *node* en el apartado *signals* al seleccionar un nodo del *SceneTree*. El patrón *Singleton* se usó para hacer instancia única y global al *StateManager* quien maneja las máquinas de estados, capacidad necesaria para el uso de las mismas ya que tener maquinas de estados para cada unidad en un videojuego donde podrían existir cientos o miles supondría una carga enorme. *Observer* a su vez se muestra principalmente en las conexiones con las interfaces de

usuario, donde se conecta el evento de selección al método de mostrar esas unidades o el cambio de un recurso con el método de mostrar ese recurso en la interfaz.

Por otra parte se realizó una implementación del patrón *Strategy* con la clase *StateMachine*, que modifica el comportamiento de otras clases (*Unit* por ejemplo) sin modificar su código.

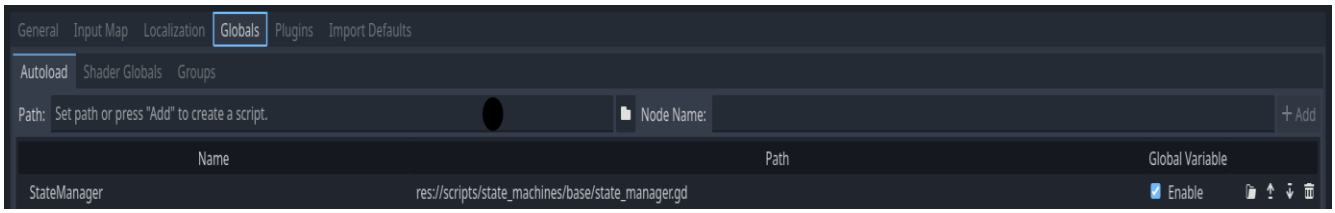

Figura 19: Uso del patrón Singleton mediante autoloads en Godot [Elaboración propia].

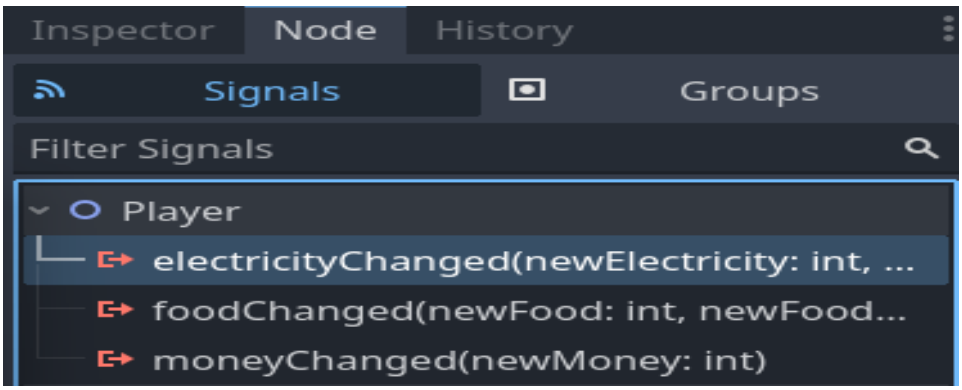

Figura 20: Panel de señales en Godot [Elaboración propia].

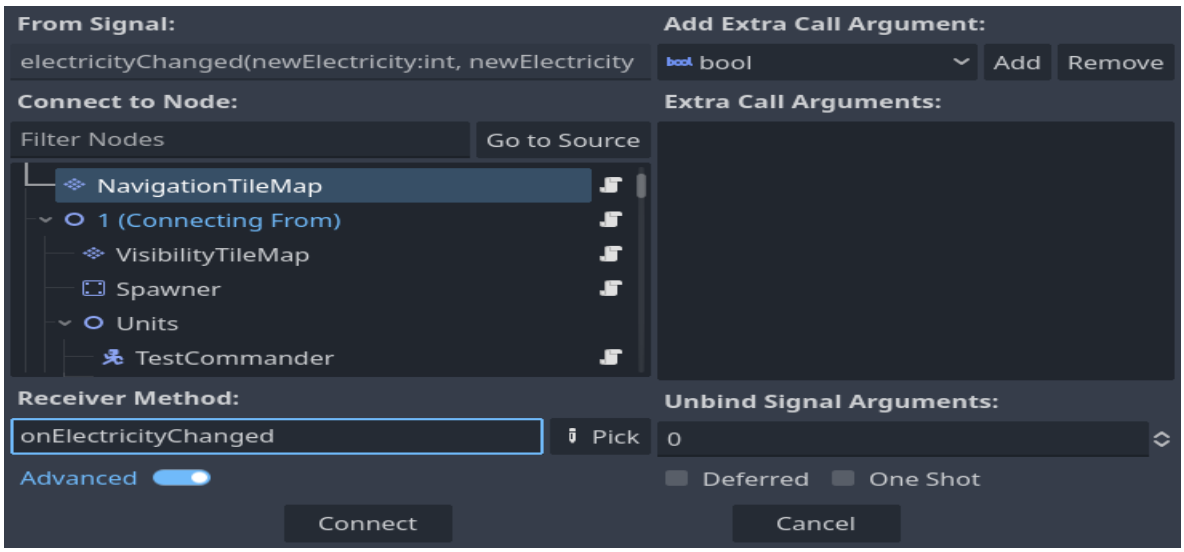

Figura 21: Panel de conexión de señales en Godot [Elaboración propia].

```
13  ##Sets up all UI connections that depend on selection
14  func connectionSetup(player : Player) -> void:
15      var selector : Selector = player.selector
16      player.electricityChanged.connect(electricityContainer.onValueChanged)
17      player.foodChanged.connect(foodContainer.onValueChanged)
18      player.moneyChanged.connect(moneyContainer.onValueChanged)
19      player.electricityChanged.emit(player.electricityUsed, player.electricityMax)
20      player.moneyChanged.emit(player.money)
21      player.foodChanged.emit(player.food, player.foodMax)
22      selector.selected.connect(unitContainer.onUnitSelection)
23      selector.userSelected.connect(unitContainer.onUserSelection)
```

Figura 22: Conexión de señales a través de código [Elaboración propia].

```
29      idle.enterFunction = func idleEnter(unit : Unit):
30          unit.campingPosition = unit.global_position
31          unit.sprite.play("idle")
32      idle.updateFunction = func idleUpdate(unit : Unit):
33          var bodies : Array[Node2D] = unit.detectionArea.get_overlapping_bodies()
34          if bodies.size() > 0:
35              unit.targetUnit = bodies[0]
36              unit.aiComponent.currentState = chase
37              return
38      idle.exitFunction = func idleExit(_unit : Unit):
39          pass
```

Figura 23: Uso del patrón Strategy en la máquina de estados de la clase Unit [Elaboración propia].

II.4.3 Tareas de Ingeniería

Las tareas ingenieriles se entienden como artefactos que materializan el proceso de creación y evolución de un sistema. Estos artefactos reflejan de forma tangible las decisiones metodológicas y técnicas adoptadas. Cada uno de estos elementos no solo evidencia el progreso y la calidad del proyecto, sino que también facilita la verificación, el mantenimiento y la comunicación entre los distintos integrantes del equipo de desarrollo, permitiendo una trazabilidad completa de las actividades realizadas. Esta perspectiva resalta que las tareas ingenieriles, al traducirse en artefactos, se constituyen en componentes fundamentales para la implementación de buenas prácticas en la ingeniería de software, favoreciendo la revisión iterativa y la mejora continua del producto final (Artefactos del modelo de desarrollo de software [Prezi], 2025). A continuación se muestran las tareas de ingeniería elaboradas en el desarrollo de la propuesta de solución

Tarea	
Numero de tarea: 1	Numero de historia de usuario: 1
Tipo de tarea: Desarrollo	Puntos estimados: 1.0
Fecha de inicio: 7/3/2025	Fecha de fin: 8/3/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de la selección exclusiva de unidades en la clase Selector. Al seleccionar una unidad se deselecciona el resto y se señala la actual.	

Tabla 49: Tarea de Ingeniería 1 [Elaboración propia].

Tarea	
Numero de tarea: 2	Numero de historia de usuario: 2
Tipo de tarea: Desarrollo	Puntos estimados: 1.5
Fecha de inicio: 8/3/2025	Fecha de fin: 10/3/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de la selección múltiple de unidades en la clase Selector . Para ello se crea un rectángulo traslúcido que encierra a las unidades, las que se encuentren dentro serán seleccionadas.	

Tabla 50: Tarea de Ingeniería 2 [Elaboración propia].

Tarea	
Numero de tarea: 3	Numero de historia de usuario: 3
Tipo de tarea: Desarrollo	Puntos estimados: 0.5
Fecha de inicio: 10/3/2025	Fecha de fin: 11/3/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de selección modificable en la clase Selector. Cambia el estado de selección de la unidad, removiéndola si está seleccionada y agregándola en caso contrario.	

Tabla 51: Tarea de Ingeniería 3 [Elaboración propia].

Tarea	
Numero de tarea: 4	Numero de historia de usuario: 4
Tipo de tarea: Desarrollo	Puntos estimados: 10.0
Fecha de inicio: 11/3/2025	Fecha de fin: 21/3/2025

Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de métodos para la manipulación del NavigationTileMap y funciones de navegación de la clase Unit.	

Tabla 52: Tarea de Ingeniería 4 [Elaboración propia].

Tarea	
Numero de tarea: 5	Numero de historia de usuario: 5
Tipo de tarea: Desarrollo	Puntos estimados: 4.0
Fecha de inicio: 21/3/2025	Fecha de fin: 25/3/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de orden de movimiento en la clase Selector. Establece los objetivos de las unidades y las ordena a moverse.	

Tabla 53: Tarea de Ingeniería 5 [Elaboración propia].

Tarea	
Numero de tarea: 6	Numero de historia de usuario: 6
Tipo de tarea: Desarrollo	Puntos estimados: 5.0
Fecha de inicio: 25/3/2025	Fecha de fin: 30/3/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de las funcionalidades para crear formación y asignar posiciones a las unidades en el FormationManager.	

Tabla 54: Tarea de Ingeniería 6 [Elaboración propia].

Tarea	
Numero de tarea: 7	Numero de historia de usuario: 7
Tipo de tarea: Desarrollo	Puntos estimados: 1.0
Fecha de inicio: 30/3/2025	Fecha de fin: 31/3/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de orden de alto en la clase Selector. Ordena a las unidades a detener su acción actual.	

Tabla 55: Tarea de Ingeniería 7 [Elaboración propia].

Tarea	
Numero de tarea: 8	Numero de historia de usuario: 8

Tipo de tarea: Desarrollo	Puntos estimados: 3.0
Fecha de inicio: 31/3/2025	Fecha de fin: 3/4/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de orden atacar en la clase Selector y comportamientos chase y attack en la clase Unit.	

Tabla 56: Tarea de Ingeniería 8 [Elaboración propia].

Tarea	
Numero de tarea: 9	Numero de historia de usuario: 9
Tipo de tarea: Desarrollo	Puntos estimados: 2.0
Fecha de inicio: 3/4/2025	Fecha de fin: 5/4/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de orden atacar-mover en la clase Selector y comportamiento engage en la clase Unit.	

Tabla 57: Tarea de Ingeniería 9 [Elaboración propia].

Tarea	
Numero de tarea: 10	Numero de historia de usuario: 10
Tipo de tarea: Desarrollo	Puntos estimados: 1.0
Fecha de inicio: 5/4/2025	Fecha de fin: 6/4/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de la clase RTSCamera y sus funcionalidades para mover la cámara con las flechas y llevando el ratón a los límites de la pantalla.	

Tabla 58: Tarea de Ingeniería 10 [Elaboración propia].

Tarea	
Numero de tarea: 11	Numero de historia de usuario: 11
Tipo de tarea: Desarrollo	Puntos estimados: 1.5
Fecha de inicio: 6/4/2025	Fecha de fin: 8/4/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	

Descripción: Implementación de interfaz barra de estado y su funcionalidad para ver el estado de la unidad principal seleccionada. Conlleva actualizar la interfaz con sus atributos en tiempo real.

Tabla 59: Tarea de Ingeniería 11 [Elaboración propia].

Tarea	
Numero de tarea: 12	Numero de historia de usuario: 12
Tipo de tarea: Desarrollo	Puntos estimados: 1.2
Fecha de inicio: 8/4/2025	Fecha de fin: 10/4/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de interfaz contenedor de unidades y su funcionalidad para asignar y seleccionar grupos de control.	

Tabla 60: Tarea de Ingeniería 12 [Elaboración propia].

Tarea	
Numero de tarea: 13	Numero de historia de usuario: 13
Tipo de tarea: Desarrollo	Puntos estimados: 0.5
Fecha de inicio: 10/4/2025	Fecha de fin: 10/4/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de funcionalidad de centrar camara en la clase RTSCamera	

Tabla 61: Tarea de Ingeniería 13 [Elaboración propia].

Tarea	
Numero de tarea: 14	Numero de historia de usuario: 14
Tipo de tarea: Desarrollo	Puntos estimados: 8.0
Fecha de inicio: 10/4/2025	Fecha de fin: 18/4/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementacion de la clase Minimap y su interfaz con la funcionalidad de mostrar unidades en el minimapa con los colores de cada jugador. El minimapa debe actualizarse en intervalos cortos sin pérdida de rendimiento notable.	

Tabla 62: Tarea de Ingeniería 14 [Elaboración propia].

Tarea

Numero de tarea: 15	Numero de historia de usuario: 15
Tipo de tarea: Desarrollo	Puntos estimados: 6.0
Fecha de inicio: 18/4/2025	Fecha de fin: 24/4/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de las clases SkillData, Skill, SkillManager y sus funcionalidades para usar habilidades. Conlleva crear la interfaz barra de habilidades y actualizar el tiempo de recarga de las habilidades.	

Tabla 63: Tarea de Ingeniería 15 [Elaboración propia].

Tarea	
Numero de tarea: 16	Numero de historia de usuario: 16
Tipo de tarea: Desarrollo	Puntos estimados: 1.0
Fecha de inicio: 24/4/2025	Fecha de fin: 25/4/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de la funcionalidad para mostrar descripción de habilidades en la interfaz barra de habilidades al poner el ratón sobre la misma.	

Tabla 64: Tarea de Ingeniería 16 [Elaboración propia].

Tarea	
Numero de tarea: 17	Numero de historia de usuario: 17
Tipo de tarea: Desarrollo	Puntos estimados: 4.0
Fecha de inicio: 25/4/2025	Fecha de fin: 29/4/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de la clase FactionManager con métodos para crear facciones, además de asignar y obtener relaciones entre facciones. Modificar la implementación de las órdenes de ataque para que haga uso de las nuevas funcionalidades.	

Tabla 65: Tarea de Ingeniería 17 [Elaboración propia].

Tarea	
Numero de tarea: 18	Numero de historia de usuario: 18
Tipo de tarea: Desarrollo	Puntos estimados: 20.0
Fecha de inicio: 29/4/2025	Fecha de fin: 18/5/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	

Descripción: Implementación de las clases FogInfo y FogOfWar con su funcionalidades para mandar los parámetros necesarios a un shader que se encarga de revelar las posiciones de las unidades del jugador. Además implementación de la clase VisibilityTileMap para hacer visible o invisible unidades dentro y fuera del efecto de la niebla. Modificaciones en la selección para impedir selección de unidades invisibles.

Tabla 66: Tarea de Ingeniería 18 [Elaboración propia].

Tarea	
Numero de tarea: 19	Numero de historia de usuario: 19
Tipo de tarea: Desarrollo	Puntos estimados: 5.0
Fecha de inicio: 18/5/2025	Fecha de fin: 23/5/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de la habilidad Build para permitir la construcción de edificios. Implementación de la funcionalidad isSpaceUsed() del NavigationTileMap para garantizar correcta posición de las edificaciones.	

Tabla 67: Tarea de Ingeniería 19 [Elaboración propia].

Tarea	
Numero de tarea: 20	Numero de historia de usuario: 20
Tipo de tarea: Desarrollo	Puntos estimados: 7.0
Fecha de inicio: 23/5/2025	Fecha de fin: 30/5/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	
Descripción: Implementación de la habilidad Train para poder entrenar unidades. Conlleva la implementación de la interfaz cola de entrenamiento y funcionalidades para guardar y modificar la cola.	

Tabla 68: Tarea de Ingeniería 20 [Elaboración propia].

Tarea	
Numero de tarea: 21	Numero de historia de usuario: 21
Tipo de tarea: Desarrollo	Puntos estimados: 4.0
Fecha de inicio: 30/5/2025	Fecha de fin: 2/6/2025
Programador responsable: Oscar de Jesús Pacheco del Pino	

Descripción: Implementación del manejo de recursos en la clase Player. Implementación de la interfaz contenedor de recursos. Modificación de descripciones de unidad y habilidades para que muestren los recursos utilizados.

Tabla 69: Tarea de Ingeniería 21 [Elaboración propia].

Conclusiones del capítulo

1. Mediante el uso de historias de usuario, estimaciones precisas de tiempo y un plan de iteraciones, se evidencia la eficiencia y robustez de la planificación, permitiendo prever y mitigar riesgos.
2. La definición de la arquitectura, la elaboración de prototipos de interfaces gráficas y las tarjetas CRC, garantiza que el sistema sea modular y escalable. Estas actividades de diseño confirman que se han tomado decisiones que priorizan la calidad técnica y la experiencia del usuario, asegurando una transición fluida a la siguiente etapa de implementación.
3. El uso de estándares de codificación y la aplicación de patrones de diseño tanto del enfoque GRASP como de los patrones de *Gang of Four*, ilustran el compromiso con las buenas prácticas en Ingeniería de Software, garantizando un producto final de alta calidad.
4. Cada etapa refleja un proceso meticuloso y orientado a resultados prácticos resultando en un videojuego de estrategia en tiempo real para contribuir a la promoción de los Objetivos de Desarrollo Sostenible. Esto destaca la pertinencia de las decisiones tomadas durante el proceso de desarrollo y respalda la posición científica adoptada, mostrando que se logró cumplir con el objetivo y resolver el problema científico planteado.

CAPÍTULO III: VALIDACIÓN DE LA SOLUCIÓN PROPUESTA

En este capítulo tiene como objetivo validar la solución propuesta mediante pruebas de software, este se estructura en dos ejes principales: (1) diseño de la validación, epígrafe que describe la necesidad e importancia de las pruebas de software y la estrategia de pruebas a emplear y (2) validación, en el cual se reflejan los resultados de la aplicación de las pruebas de software.

III.1 Diseño de la validación

Este epígrafe trata la importancia vital que tienen las pruebas de software enmarcadas en la fase de refactorización y pruebas de la metodología XP y presenta la estategia de pruebas utilizada en el ciclo de desarrollo del videojuego Sustainable Conquest.

Las pruebas de software son el proceso sistemático mediante el cual se evalúa y verifica la calidad y correcto funcionamiento de un sistema informático, asegurando que cumpla tanto con los requerimientos técnicos como con las expectativas del usuario final. Durante este proceso se aplican diferentes técnicas, que van desde las pruebas unitarias, que se centran en validar cada componente individual del sistema, hasta las pruebas de integración y de aceptación, que confirman la correcta interacción entre módulos y la conformidad con las especificaciones del proyecto. La combinación de pruebas manuales y automatizadas permite identificar errores de forma temprana, optimizar los tiempos de corrección y garantizar la estabilidad y seguridad del software a lo largo de su ciclo de vida, lo que es especialmente fundamental en entornos de desarrollo ágil e iterativo (IBM, 2025; Turrado, 2020).

III.1.1 Estrategia de pruebas

En el caso de Sustainable Conquest siguiendo los estándares de la metodología XP se realizaron pruebas funcionales unitarias , de aceptación a nivel beta y de índice de aceptación. Se utilizaron las primeras para validar funcionalidades específicas de alta importancia o complejidad, de forma automatizada y las últimas para validar el producto como un todo desde el punto de vista del usuario final con una muestra de diez personas En caso de encontrarse una no conformidad se documenta y se trabaja en el período de refactorización entre iteraciones.

III.1.1.1 Pruebas unitarias

Las pruebas unitarias son una metodología de verificación esencial en el desarrollo de software, orientada a evaluar de forma aislada la funcionalidad de la unidad mínima de código, ya sea una función, método o módulo. Este proceso, que se inscribe dentro de las pruebas de caja blanca, consiste en ejecutar el código con conjuntos específicos de entradas y comparar las salidas reales con las esperadas, lo que permite identificar y corregir errores en etapas tempranas del ciclo de desarrollo. Además, al utilizar dobles de prueba para aislar la unidad bajo pruebas de las dependencias externas, las pruebas unitarias promueven la modularidad, facilitan el mantenimiento del sistema y contribuyen a garantizar la calidad global del software, especialmente en entornos de integración continua y metodologías ágiles (Amazon Web Services, n.d.; Testing IT, 2022).

III.1.1.2 Pruebas de aceptación

Las pruebas de software de aceptación son una fase crítica del proceso de validación en la que se verifica que el sistema cumple con los requerimientos y expectativas definidos por el cliente o usuario final. Conocidas también como UAT (*User Acceptance Testing*), estas pruebas se llevan a cabo en entornos que emulan condiciones reales de uso, permitiendo a los propios usuarios o a sus representantes confirmar que la aplicación es funcional, usable y responde a las necesidades del negocio. Durante este proceso se identifican posibles discrepancias o defectos que, pese a haber sido corregidos en etapas previas, podrían afectar la experiencia final, asegurando así que el producto cumple tanto con las especificaciones técnicas como con los criterios de aceptación acordados contractualmente. Este enfoque centrado en el usuario garantiza que el software no solo sea técnicamente robusto, sino también alineado con las expectativas del mercado y de sus usuarios, consolidando la confianza tanto en el producto como en el proceso de desarrollo (Nimblework, 2024; QAMakers, 2023).

III.1.1.2 Pruebas de índice de aceptación

El índice de satisfacción es una métrica que cuantifica la percepción que tienen los usuarios o clientes sobre la calidad y desempeño de un producto, servicio o experiencia. Esta evaluación se realiza generalmente mediante encuestas basadas en escalas tipo Likert, en las que se pide a los usuarios que califiquen diferentes aspectos (por ejemplo, usabilidad, desempeño, contenido, atención al cliente) en una escala numérica. Uno de los indicadores comunes derivados de este proceso es el *Customer Satisfaction Score* (CSAT), el cual se calcula como

el porcentaje de respuestas positivas (normalmente aquellas valoradas con 4 o 5 en una escala de 1 a 5) respecto al total de respuestas. Este índice posibilita el monitoreo de la experiencia del usuario en tiempo real, convirtiéndose en un elemento clave para la toma de decisiones estratégicas orientadas a la mejora continua del producto o servicio (Zea, Morán Chiquito, Vergara Romero, & Jimber del Río, 2022; Arteaga Díaz, Escobar Montero, & Piñeros Guzmán, 2021).

III.2 Validación

En este epígrafe se describe la aplicación de las pruebas de software a nivel unitario, de aceptación y de índice de aceptación, además de los resultados de cada una de estas y cómo se trataron para garantizar la validación de las funcionalidades de la propuesta de solución.

III.2.1 Aplicación de pruebas unitarias

Las pruebas unitarias se realizaron de forma automatizada utilizando el *framework Godot Unit Testing* (GUT), estándar para realizar pruebas unitarias utilizando el motor Godot. Como ejemplo se muestran las pruebas a las funcionalidades de la clase *Player*, mostrándose casos de prueba para la funcionalidad *hasEnoughResources()* que retorna verdadero si el jugador tiene los recursos suficientes para soportar una producción con los recursos pasados por parámetro (créditos, provisiones y electricidad en ese orden). Obteniéndose resultados exitosos en las 8 pruebas diseñadas para esta funcionalidad

```
1   extends GutTest
2
3
4   class TestHasEnoughResources extends GutTest:
5       var player : Player = Player.new()
6
7       func test_noCostResourcesShouldPass()-> void:
8           assert_eq(player.hasEnoughResources(0, 0, 0), true, "No cost did not pass")
9
10
11      func test_higherCostThanAvailableShouldnotPass()-> void:
12          assert_eq(player.hasEnoughResources(1, 1, 1), false, "Higher cost than available passed")
13
14
15      func test_lowerCostThanAvailableShouldPass() -> void:
16          var testPlayer : Player = Player.new()
17          testPlayer.money = 100
18          testPlayer.electricityMax = Global.maxElectricity
19          testPlayer.foodMax = Global.maxFood
20          assert_eq(testPlayer.hasEnoughResources(50, Global.maxFood - 1, Global.maxElectricity - 1), true, "LowerCost did not pass")
```

Figura 24: Casos de prueba para funcionalidad hasEnoughResources() de la clase Player [Elaboración propia].

```
                    Finished                    0.0s
==============================================
= Run Summary
==============================================

---- Totals ----
Scripts            4
Tests              8
  Passing          8
Asserts            8
Time               0.005s


---- All tests passed! ----

Tests     100%   res://tests
Scripts   100%   test_player.gd.TestGetElectricity

Compact
```

Figura 25: Resultado de pruebas para la clase Player [Elaboración propia].

Todas las funcionalidades probadas de esta forma pasaron las pruebas de forma exitosa, que no significa que no puedan existir defectos, ya que la interoperabilidad entre módulos puede causar problemas, lo que lleva a la necesidad de la aplicación de otro nivel de pruebas.

III.2.3 Aplicación de pruebas de aceptación

Las pruebas de aceptación se realizaron a nivel beta, lo que significa que la realizan usuarios finales en sus condiciones propias, no preparadas ni supervisadas por los desarrolladores. Esto garantiza una mayor cantidad de variables expuestas a probarse y su finalización determina si el producto realmente está listo para lanzarse o no.

Para su realización se tomó una muestra de estudiantes de la carrera Ingeniería en Ciencias Informáticas a los que se les distribuyó el videojuego Sustainable Conquest sin manual ni explicaciones de cómo jugar para probar la utilidad de los tutoriales dentro del videojuego.

Nombre y Apellidos	Año
Cristian Medina Liu	4
Dariel Angel Arró Moreno	4
Dairon Luis Naranjo Almanza	3
Derik González Martín	4

Leinier Garcés Villa	4
Melissa Treto Blanco	4
Pablo Ariel Rivero Bencomo	4
Wilder Navarro Sorís	4
Yoel Rodríguez Garcia	4
Yslen Marturelo Lorenzo	4

Tabla 70: Listado de estudiantes probadores beta [Elaboración propia].

Luego de realizar tres iteraciones, no se encontraron no conformidades. Por lo que se evidencia que el producto es válido y está listo para lanzarse .

III.2.3 Aplicación de pruebas de índice de aceptación

Para la aplicación de esta prueba se tomó la misma muestra que realizó las pruebas beta y se les entregó un cuestionario diseñado de la siguiente forma:

Cuestionario de Satisfacción del Usuario – Sustainable Conquest
Instrucciones: Por favor, responde cada pregunta considerando la siguiente escala de Likert: 1: Muy insatisfecho/a 2: Insatisfecho/a 3: Neutral 4: Satisfecho/a 5: Muy satisfecho/a
Sección 1: Experiencia General y Jugabilidad
Jugabilidad General: ¿Qué tan satisfecho/a estás con la jugabilidad general del videojuego?
Control y Mecánicas: ¿Qué tan intuitivas te resultan las mecánicas de control (selección de unidades, órdenes de movimiento, interacciones) en el juego?
Desafíos y Equilibrio: ¿Consideras que el nivel de desafío y el equilibrio entre acción y estrategia son adecuados para ofrecer una experiencia entretenida y retadora?
Sección 2: Interfaz y Diseño Visual

Usabilidad e Interfaz: ¿Qué tan clara e intuitiva es la interfaz del videojuego? ¿Qué tan útil te han resultado los tutoriales y elementos de ayuda integrados?
Estética y Diseño Gráfico: ¿Qué tan atractivo y coherente encuentras el diseño visual y la estética general del videojuego?
Sección 3: Rendimiento Técnico
Rendimiento del Sistema: ¿Estás satisfecho/a con el desempeño técnico del juego en términos de tiempos de carga, estabilidad y fluidez durante la experiencia de juego?
Sección 4: Contenido Educativo y Relación con los ODS
Integración de Contenido Educativo: ¿Qué tan claro y efectivo es el contenido relacionado con los Objetivos de Desarrollo Sostenible (ODS) dentro del videojuego?
Impacto Educativo: ¿Crees que el contenido del juego te ha motivado a informarte o reflexionar sobre temas de sostenibilidad?
Balance entre Entretenimiento y Educación: ¿Consideras que el juego logra un buen equilibrio entre ser entretenido y cumplir con su función educativa en cuanto a la promoción de los ODS?
Sección 5: Satisfacción Global y Recomendación
Satisfacción Global: En general, ¿qué tan satisfecho/a estás con tu experiencia en Sustainable Conquest?
Recomendación: ¿Recomendarías este videojuego a otras personas interesadas en videojuegos educativos o en sostenibilidad? (Opciones: Sí / No)

Tabla 71: Cuestionario de Satisfacción del Usuario - Sustainable Conquest [Elaboración Propia].

Al aplicarlo se obtuvieron resultados abrumadoramente positivos con todos los usuarios recomendando probar el videojuego, además 108 de 110 respuestas fueron positivas con

puntuaciones de 4 o más y solamente 2 puntuaciones neutrales referentes a lo claras e intuitivas que son las interfaces del videojuego, identificando una posible área de mejora. Tomando estos resultados se aplica la fórmula del CSAT resultando en un índice de aceptación de 98.18%, lo cual se considera extremadamente alto.

Conclusiones del capítulo

1. El diseño estructurado de la validación permite asegurar una alta calidad en el mismo proceso siguiendo las buenas prácticas estandarizadas por la metodología *Extreme Programming*.
2. La aplicación de pruebas unitarias automatizadas garantiza el correcto funcionamiento de los módulos sin dependencias externas, mientras que las de aceptación validan al videojuego como un todo desde el punto de vista de usuarios finales.

CONCLUSIONES FINALES

1. Los videojuegos de estrategia en tiempo real (RTS) son herramientas versátiles que combinan entretenimiento y educación, demostrando su potencial para fomentar habilidades cognitivas como el pensamiento crítico, la resolución de problemas y la gestión del tiempo, teniendo un papel relevante en la promoción de los ODS al permitr que los jugadores experimenten con estrategias que equilibran el desarrollo económico, social y ambiental.
2. Se aplicó un proceso meticuloso y orientado a resultados práctico basado en la metodología *Extreme Programming*, resultando en un videojuego de estrategia en tiempo real que contribuye a la promoción de los Objetivos de Desarrollo Sostenible. Esto destaca la pertinencia de las decisiones tomadas durante el proceso de desarrollo y respalda la posición científica adoptada, mostrando que se logró cumplir con el objetivo y resolver el problema científico planteado
3. Se realizó un estricto proceso de validación mediante la aplicación de pruebas unitarias y de aceptación para garantizar la calidad y la funcionalidad correcta del videojuego Sustainable Conquest.

RECOMENDACIONES

1. Evaluar el impacto en la formación complementando la validación actual con estudios de impacto educativo, donde se mida el conocimiento y cambio de actitudes de los usuarios (por ejemplo, antes y después de jugar) respecto a los Objetivos de Desarrollo Sostenible. Se sugiere desarrollar una metodología mixta que combine análisis cuantitativo y cualitativo.
2. Ampliar la narrativa y las mecánicas mediante la incorporación de módulos de contenido específicos para distintos ODS. Por ejemplo, además del enfoque existente en sostenibilidad ambiental, se podría incorporar nuevos escenarios, misiones y desafíos que aborden otros ODS.
3. Promover la presentación y discusión del videojuego en conferencias, ferias y seminarios sobre educación, sostenibilidad y desarrollo tecnológico. Esto facilitará la retroalimentación externa y abrirá nuevas oportunidades de colaboración y mejora.
4. Explorar la incorporación de elementos de gamificación más avanzados, como sistemas de recompensa basados en logros sostenibles, desafíos colaborativos y rankings dinámicos, que incentiven un aprendizaje activo y una mayor inmersión en los temas de desarrollo sostenible.
5. Integración de técnicas de inteligencia artificial para adaptar dinámicamente la dificultad, la narrativa y los retos del juego según el perfil y comportamiento del usuario, favoreciendo un aprendizaje personalizado y evolutivo.
6. Actualización constante del contenido educativo relacionado con los ODS para asegurarse de que permanece alineado con las nuevas investigaciones, políticas y compromisos internacionales. Esto garantiza que el videojuego siga siendo una herramienta actualizada y relevante en el ámbito de la educación para el desarrollo sostenible.
7. Uso del videojuego como herramienta didáctica: Proponer su implementación como recurso complementario en planes de estudio de carreras relacionadas con sostenibilidad, ingeniería y ciencias de la computación. Esto podría incluir la elaboración de materiales de apoyo, guías docentes y módulos de capacitación para el uso efectivo del juego en contextos formales y no formales.

8. Optimizar la experiencia en dispositivos variados como porte a dispositivos móviles o la aplicación de técnicas avanzadas (por ejemplo, en el manejo de shaders o en la inteligencia artificial de las unidades) para que el videojuego mantenga un rendimiento adecuado incluso en hardware de menor potencia. Esto ampliaría la accesibilidad y la base de usuarios potenciales.
9. Refinar las interfaces gráficas mediante pruebas de usabilidad; prestar especial atención a la claridad de los tutoriales iniciales y a la retroalimentación visual y sonora en respuesta a las acciones del jugador para garantizar una experiencia intuitiva.

REFERENCIAS BIBLIOGRÁFICAS

World Commission on Environment and Development (WCED). (1987). Our common future. Oxford University Press.

Naciones Unidas. (2015). Transformar nuestro mundo: la Agenda 2030 para el Desarrollo Sostenible. https://unctad.org/system/files/official-document/ares70d1_es.pdf

Naciones Unidas. (2020). Objetivos de Desarrollo Sostenible: 17 objetivos para transformar nuestro mundo. https://www.un.org/sustainabledevelopment/es/

Stiglitz, J. E., Sen, A., & Fitoussi, J. P. (2018). Report by the Commission on the Measurement of Economic Performance and Social Progress. OECD.

European Union. (2024). Directive (EU) 2024/825 on empowering consumers for the green transition through better protection against unfair practices and better information. Official Journal of the European Union. https://eur-lex.europa.eu

UNESCO. (2017). Educación para los Objetivos de Desarrollo Sostenible: objetivos de aprendizaje. https://www.unesco.org/es/articles/educacion-para-los-objetivos-de-desarrollo-sostenible-objetivos-de-aprendizaje

UNESCO. (2020). Educación para el desarrollo sostenible: hoja de ruta. https://www.unesco.org/es/articles/educacion-para-el-desarrollo-sostenible-hoja-de-ruta

Global Education Monitoring Report Team. (2022). Global Education Monitoring Report 2021/2: Non-State Actors in Education: Who Chooses? Who Loses?. UNESCO. https://en.unesco.org/gem-report/non-state_actors

UNESCO. (2023). Los adolescentes tienen el doble de probabilidad de no ir a la escuela que los niños en edad escolar. https://www.unesco.org/es/articles/los-adolescentes-tienen-el-doble-de-probabilidad-de-no-ir-la-escuela-que-los-ninos-en-edad-escolar

UNESCO. (2023). Informe de Seguimiento de la Educación en el Mundo 2023: Cuba. https://www.unesco.org/gem-report/es

Banco Mundial. (2025). Programa de Educación para América Latina y el Caribe. Recuperado el 17 de abril de 2025, de https://www.bancomundial.org/es/programs/educacion-america-latina-caribe

UNICEF. (2020). La accesibilidad de la educación a distancia: Un informe sobre la brecha digital durante el COVID-19. https://coronavirus.onu.org.mx/al-menos-una-tercera-parte-de-los-ninos-en-edad-escolar-en-el-mundo-no-tuvo-acceso-a-educacion-a-distancia-durante-el-cierre-de-las-escuelas-por-covid-19

Constitución de la República de Cuba. (2019). Artículo 73: Derecho a la educación. Gaceta Oficial No. 5 Extraordinaria.

Banco Mundial. (2023). Indicadores de Desarrollo Mundial: Educación en Cuba. https://datos.bancomundial.org/indicador/SE.ADT.LITR.ZS?locations=CU

Mesa-Lago, C. (2022). La economía cubana: Entre el embargo y la pandemia. Revista de Ciencia Política, 42(1), 45-68.

Grupo Nacional para la Implementación de la Agenda 2030 (Ministerio de Economía y Planificación de la República de Cuba). (2021). Informe Nacional Voluntario Cuba 2021 sobre la Agenda 2030. https://www.mep.gob.cu/es/documento/informe-nacional-voluntario-cuba-2021-sobre-la-agenda-2030

Plass, J. L., Homer, B. D., & Kinzer, C. K. (2020). Foundations of game-based learning. Educational Psychologist, 50(4), 258–283. https://doi.org/10.1080/00461520.2015.1122533

Flood, S., Cradock-Henry, N. A., & Blackett, P. (2024). A collaborative adaptation game for promoting climate action: Minions of Disruptions™. Geoscience Communication, 7(1), 167–189. https://doi.org/10.5194/gc-7-167-2024

Hawlitschek, F., Joeckel, S., & Wittstock, M. (2022). Gameful Learning for a More Sustainable World: Measuring the Effect of Design Elements on Long-Term Learning Outcomes in Correct Waste Sorting. Business & Information Systems Engineering, 64(4), 459–482. https://doi.org/10.1007/s12599-021-00731-x

Wu, J. S., & Lee, J. J. (2020). Climate change games as tools for education and engagement. Nature Climate Change, 10(5), 526–532. https://doi.org/10.1038/s41558-020-0740-1

11 Bit Studios. (2024). Frostpunk 2: A deep dive into ethical city-building. https://www.11bitstudios.com/games/frostpunk-2

Paradox Interactive. (2016). Stellaris [Videojuego]. PC.

Relic Entertainment. (2021). Age of Empires IV [Videojuego]. Xbox Game Studios.

Chen, J., Smith, T., & Alvarez, R. (2022). Adaptive difficulty in video games: A machine learning approach. IEEE Transactions on Games, 15(2), 1–14.

Johnson, D., & Lee, K. (2021). The psychology of game design: How feedback loops shape player behavior. Computers in Human Behavior, 114, 106557.
Rogers, S. (2020). Level up! The guide to great video game design (3.ª ed.). Wiley.

Programa de las Naciones Unidas para el Desarrollo. (2021). Informe sobre Desarrollo Humano 2021/2022: Tiempos inciertos, vidas inestables.https://hdr.undp.org/informe-sobre-desarrollo-humano-2021-22

Elkington, J. (2021). Green Swans: The Coming Boom in Regenerative Capitalism. Fast Company Press.ISBN-13: 978-1639080021

Sachs, J. D., Kroll, C., Lafortune, G., Fuller, G., & Woelm, F. (2021). *Sustainable Development Report 2021: The Decade of Action for the Sustainable Development Goals*. Cambridge University Press. https://doi.org/10.1017/9781009106559

Sterling, S. (2016). *Educating for Sustainability: Principles and Practices for Teachers*. Routledge. ISBN-13: 978-1138470955

Deterding, S., Zagal, J. P., & Björk, S. (2020). Game design elements in non-game contexts: A framework for gamification. *Proceedings of the ACM on Human-Computer Interaction, 4*(CSCW2), 1–22.

European Council. (2022). *Digital education action plan 2021-2027: Resetting education and training for the digital age*. https://education.ec.europa.eu/sites/default/files/document-library-docs/deap-communication-sept2020_en.pdf

Plass, J. L., Mayer, R. E., & Homer, B. D. (2020). *Handbook of game-based learning*. MIT Press. https://mitpress.mit.edu/9780262043380/handbook-of-game-based-learning/

UNESCO. (2021). *Reimagining our futures together: A new social contract for education*. https://unesdoc.unesco.org/ark:/48223/pf0000379707

Lee, S., Kim, J., & Park, T. (2022). Strategic resource allocation in real-time strategy games: A framework for adaptive decision-making. *Journal of Gaming & Virtual Worlds, 14*(3), 215-234.

WIRED (2021). The Fall and Rise of Real-Time Strategy Games. https://www.wired.com/story/fall-and-rise-real-time-strategy-games/

Martínez, L., Gimenes, M., & Lambert, E. (2023). Video games and board games: Effects of playing practice on cognition. PLoS ONE, 18(3), e0283654. https://doi.org/10.1371/journal.pone.0283654

Richard, G. T., Giri, S., & Warschauer, M. (2020). Video games and environmental learning: A review of recent literature. *Computers & Education, 158*, 103991.

Zagal, J. P., Deterding, S., & Björk, S. (2022). Moral dilemmas and decision-making in post-apocalyptic games: A case study of Frostpunk. *Games and Culture, 17*(6), 789–806.

Pressman, R. S., & Maxim, B. R. (2021). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The definitive guide to Scrum: The rules of the game*. https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf

Beck, K. (1999). *Extreme Programming Explained: Embrace Change*. Addison-Wesley.

Cockburn, A. (2020). Agile Software Development: The Cooperative Game (2nd ed.). Addison-Wesley.

Schwaber, K., Sutherland, J. (2020). *The Scrum Guide*. Scrum.org.

Gregory, J. (2020). Game engine architecture (3rd ed.). CRC Press.

Hutchins, B. (2021). The art of game design: A book of lenses (2nd ed.). CRC Press.

Godot Engine. (2023). *Godot 4.3 Release Notes*. https://godotengine.org

Godot Engine. (2023). GDScript: A Python-like language for Godot. https://docs.godotengine.org/en/stable/getting_started/scripting/gdscript/gdscript_basics.html

Linietsky, J., & Manzur, A. (2020). Godot Engine: The complete guide to creating games with Godot. Packt Publishing.

Sebesta, R. W. (2020). Concepts of programming languages (12th ed.). Pearson Education.

Sommerville, I. (2021). Software engineering (10th ed.). Pearson Education.

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2021). Compilers: Principles, techniques, and tools (2nd ed.). Pearson.

Morris, D. (2021). Programming paradigms: Object-oriented, functional, and beyond. Springer.

Visual Paradigm. (2023). Visual Paradigm: The all-in-one CASE tool for software development. https://www.visual-paradigm.com/

Fowler, M. (2018). UML distilled: A brief guide to the standard object modeling language (3rd ed.). Addison-Wesley.

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. https://git-scm.com/book/en/v2

Loeliger, J., & Ponuthorai, P. K. (2022). Version Control with Git (3rd ed.). O'Reilly Media.Amaz

GitHub, Inc. (2022). GitHub Projects documentation. https://docs.github.com/en/github/managing-your-work-on-github/about-projects

Coppola, E. (2023). Dialogic Documentation. https://github.com/coppolaemilio/dialogic

Godot Engine Documentation. (2023). About Plugins. https://docs.godotengine.org

Godot Forums. (2023). Dialogic Discussions. https://forum.godotengine.org

Godot Engine. (2023). Documentación Oficial: SceneTree y Nodos. https://docs.godotengine.org

Cohn, M. (2020). Agile Estimating and Planning (2nd ed.). Prentice Hall.

Paasivaara, M., & Kruchten, P. (Eds.). (2020). Agile Processes in Software Engineering and Extreme Programming – Workshops: XP 2020 Workshops, Copenhagen, Denmark, June 8–12, 2020, Revised Selected Papers. Springer Cham. https://doi.org/10.1007/978-3-030-58858-8

Visure Solutions. (2024). Requisitos no funcionales: tipos, ejemplos y enfoques. https://visuresolutions.com/es/requirements-management-traceability-guide/non-functional-requirements/

Echavarría-Flórez, I. S., & Restrepo-Calle, F. (2020). Software Readability Metrics: A Systematic Literature Review. Revista Facultad de Ingeniería, 29(54), e11756. https://doi.org/10.19053/01211129.v29.n54.2020.11756

Vale, G., Correia, F. F., Guerra, E. M., de Oliveira Rosa, T., Fritzsch, J., & Bogner, J. (2022). Designing Microservice Systems Using Patterns: An Empirical Study on Quality Trade-Offs. arXiv preprint arXiv:2201.03598. https://doi.org/10.48550/arXiv.2201.03598

Richards, M., & Ford, N. (2020). Fundamentals of Software Architecture: An Engineering Approach. O'Reilly Media.

Fluent C++. (2021, 23 de junio). GRASP: 9 Must-Know Design Principles for Code. Recuperado de https://www.fluentcpp.com/2021/06/23/grasp-9-must-know-design-principles-for-code/

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (2021). Design Patterns: Elements of Reusable Object-Oriented Software (25th Anniversary Edition). Addison-Wesley Professional.

R3crea. (2022, 6 de octubre). Ventajas y desventajas de los objetivos de desarrollo sostenible. Recuperado de https://r3crea.com/innovacion/ventajas-y-desventajas-de-los-objetivos-de-desarrollo-sostenible/

EcologíaVerde. (2024, 4 de junio). Ventajas y desventajas del desarrollo sostenible. Recuperado de https://www.ecologiaverde.com/ventajas-y-desventajas-del-desarrollo-sostenible-319.html

Artefactos del modelo de desarrollo de software [Prezi]. (2025, 28 de mayo). Prezi. Recuperado el 28 de mayo de 2025, de https://prezi.com/p/pb5stdaurzyg/artefactos-del-modelo-de-desarrollo-de-software/

IBM. (2025). ¿Qué son las pruebas de software? IBM. Recuperado el 28 de mayo de 2025, de https://www.ibm.com/mx-es/topics/software-testing

Turrado, J. (2020, 10 de marzo). ¿Qué son las pruebas de software? CampusMVP. Recuperado el 28 de mayo de 2025, de https://www.campusmvp.es/recursos/post/que-son-las-pruebas-de-software.aspx

Amazon Web Services. (2025, 28 de mayo). ¿Qué son las pruebas unitarias? Recuperado el 28 de mayo de 2025, de https://aws.amazon.com/es/what-is/unit-testing/

Testing IT. (2022, 11 de julio). ¿Qué son las pruebas unitarias de software? Recuperado el 28 de mayo de 2025, de https://www.testingit.com.mx/blog/pruebas-unitarias-de-software

Nimblework. (2024, 26 de abril). Pruebas de aceptación: El qué y por qué. Recuperado el 28 de mayo de 2025, de https://www.nimblework.com/es/agile/pruebas-de-aceptacion/

QAMakers. (2023, 15 de diciembre). Pruebas de aceptación de software: Garantizando la excelencia en el desarrollo. Recuperado el 28 de mayo de 2025, de https://www.qamakers.com/2023/12/pruebas-de-aceptacion-de-software.html

Arteaga Díaz, Y. M., Escobar Montero, J. P., & Piñeros Guzmán, Y. H. (2021). Instrumentos de evaluación de calidad y satisfacción del usuario en instituciones prestadoras de servicios de salud a nivel de Latinoamérica: Una revisión bibliográfica. Corporación Universitaria Iberoamericana.

Zea, M., Morán Chiquito, D., Vergara Romero, A., & Jimber del Río, J. A. (2022). Modelos de satisfacción al cliente: Un análisis de los índices más relevantes: Customer satisfaction models: An analysis of the most relevant indices. RES NON VERBA REVISTA CIENTÍFICA, 12(2), 146–178. https://doi.org/10.21855/resnonverba.v12i2.735